Lower Bounds for Garbled Circuits from Shannon-Type Information Inequalities*

Jake Januzelli† Mike Rosulek† Lawrence Roy‡

May 16, 2025

Abstract

We establish new lower bounds on the size of practical garbled circuits, which hold against any scheme satisfying the following simple properties: (1) Its security is based on symmetric-key cryptography only. More formally, security holds in Minicrypt, a model in which a random oracle is the only available source of cryptography. (2) The evaluation algorithm makes non-adaptive queries to the random oracle. (3) The evaluation algorithm "knows" which of its oracle queries are made by which other input combinations. These restrictions are reasonable for garbling single gates. In particular, unlike prior attempts at lower bounds, we make no assumptions about the internal behavior of the garbling algorithms — *i.e.*, how it uses random oracle outputs and wire labels to compute the garbled gate, etc.

We prove separate lower bounds depending on whether the scheme uses the free-XOR technique (Kolesnikov & Schneider, ICALP 2008). In the free-XOR case, we prove that a garbled AND-gate requires 1.5λ bits; thus, the garbling scheme of Rosulek & Roy (Crypto 2022) is optimal. In the non-free-XOR case, we prove that a garbled AND-gate requires 2λ bits and a garbled XOR-gate requires λ bits; thus, the garbling scheme of Gueron, Lindell, Nof, and Pinkas (CCS 2015) is optimal.

We prove our lower bounds using tools from information theory. A garbling scheme can be characterized as a joint distribution over various quantities: wire labels, garbled gate information, random oracle responses. We show that different properties of a garbling scheme imply certain Shannon-type information inequalities about this distribution. We then use an automated theorem prover for Shannon-type inequalities to prove that our inequalities imply lower bounds on the entropy—hence, size—of the garbled gate information.

1 Introduction

Garbled circuits are a cryptographic technique used for constant-round MPC and many other applications. A garbled circuit is a special kind of encryption of a circuit. Given certain "keys" that encode an input to that circuit, it is possible to obtain keys that encode the corresponding output of that circuit. This evaluation process can be done blindly, without knowing the choice of input — hence the utility of garbled circuits for cryptography.

Optimizing the size of garbled circuits. Modern methods for circuit garbling use fast, hardware-accelerated AES as the main cryptographic building block. As a result, the computational cost of circuit garbling is low. The bottleneck for most applications of garbling is rather the

*First 2 authors partially supported by NSF award 2150726.

†Oregon State University, {januzelj,rosulekm}@oregonstate.edu

‡Aarhus University, ldr709@gmail.com

communication cost of sending the garbled circuit object. Since Yao's first construction of garbled circuits (folklore, but usually attributed to [Yao86]), there has been a long line of improvements to the size of garbled circuits. Beaver, Micali & Rogaway [BMR90] introduced what is now called the point-and-permute technique to make the cost of a garbled gate 4λ bits, where λ is the security parameter. Naor, Pinkas & Sumner [NPS99] introduced the row-reduction technique, which reduced the cost further to 3λ bits. Later, more advanced forms of row reduction [PSSW09, GLNP15] were proposed to further reduce the cost to 2λ. Kolesnikov & Schneider [KS08] introduced the free-XOR technique, which allowed XOR gates in the circuit to be free. Zahur, Rosulek & Evans [ZRE15] showed how to make advanced row-reduction compatible with free-XOR, resulting in a scheme with free XOR gates and AND gates that cost only 2λ bits.

The current state of the art for garbled circuits is due to Rosulek & Roy [RR21], whose scheme supports free XOR gates and requires only 1.5λ bits per AND gate. Given the long history of improvements to garbling methods, a natural question is: *what is the optimal size of a garbled circuit?*

Lower bounds for garbled circuits. Zahur, Rosulek & Evans [ZRE15] proved a lower bound on garbled circuit size, for a class of garbling methods that they called *linear garbling*. This class of garbling methods captured all techniques that were known at the time; essentially, the garbling and evaluation algorithms treat blocks of λ bits as elements in a ring and manipulate them via only (fixed) linear combinations. They showed that 2λ bits are necessary to garble an AND gate via a linear garbling scheme.

Soon after, several works [KKS16, BMR16, WM17] proposed "non-linear" garbling methods that bypassed this lower bound, resulting in AND gates with a cost of only λ bits. However, these constructions were limited in that they could garble only a single AND gate in isolation, not an entire circuit.

The leading construction of Rosulek & Roy [RR21] can garble an entire circuit, such that XOR-gates are free and AND-gates cost $1.5\lambda + O(1)$ bits. It uses two techniques that are outside of the *linear garbling* model: First, it divides wire labels into two slices of size $\lambda/2$, and performs separate linear operations on the two slices. Second, the choice of which linear operation to perform is made at the time of evaluation, whereas in the linear garbling model this choice can depend only on the so-called "color bits" of the wire labels.

Some limited lower bounds have been proven that incorporate the techniques of [RR21]. Baek and Kim [BK24] showed that slicing wire labels into more than 2 pieces cannot lead to a reduction in garbled circuit size. Fan, Lu, and Zhou generalized the three-halves construction further and proved matching lower bounds [FLZ24]. Li, Guo and Wang propose a model for garbling using only a pseudorandom function, and show lower bounds on communication and number of PRF calls made by the garbler [LGW24]. Xu, Hu, and Xu propose a "bitwise" model of garbled circuits and show lower bounds in both free-XOR and non free-XOR cases [XHX24].

Unfortunately, all of these lower bounds make structural assumptions on the internal behavior of the garbling scheme. First, they make assumptions about the random oracle queries made by the evaluator. For example, there are some queries that can be made by exactly two evaluator input combinations, but it is assumed none can be in exactly three. This is an intuitive property but imposed as a restriction on the garbling scheme. (By comparison, we *prove* that oracle queries satisfy such properties — see Section 5.4, Section 5.5). Second, the lower bounds make assumptions about how the algorithms compute their outputs, as a function of the inputs and random oracle responses:

1. Multiple works assume parts of the scheme to be linear or otherwise restrict the garbler to

known techniques like slicing/dicing [BK24, LGW24, FLZ24].

2. The PRF-based model in [LGW24] assumes structural properties on how the PRF is invoked (e.g, each PRF is keyed with an input label).

3. Two models are proposed in [XHX24]: the first assumes linearity, but the second appears at first to be quite powerful, allowing the garbler to use any *multi-variable* polynomial function of the input wire labels. However, the proof sketch proceeds by bounding the rank of a matrix, suggesting that some linearity remains present.

The "dream" lower bound. A more convincing "dream" lower bound for garbling would apply to any garbling scheme in **Minicrypt** [Imp95]. Minicrypt refers to a world where only symmetric-key cryptography (OWFs, PRGs, etc) is possible. Technically, it is a model in which adversaries are computationally unbounded but make polynomially-many queries to a random oracle. Minicrypt is the standard way of reasoning about what is possible from symmetric-key cryptography alone. Such a restriction on a garbling scheme is necessary for a fine-grained lower bound such as ours, because there exist schemes that use heavy machinery like fully homomorphic encryption or obfuscation (which do not exist in Minicrypt) to garble gates with constant (amortized) size. Note that a lower bound against Minicrypt constructions applies also to schemes whose security is based on *standard-model* assumptions about symmetric-key primitives, since a random oracle also implies these assumptions.

1.1 Overview of Our Results

In this work we establish new lower bounds for garbled circuits, for the broadest class of garbling schemes so far.

The lower bound model. Our bounds apply to Minicrypt garbling schemes that satisfy the following additional requirements:

- The evaluation algorithm makes only **non-adaptive queries** to the random oracle. Modern garbling schemes indeed make adaptive oracle queries, but only when garbling a large circuit. We know of none that make adaptive queries when *garbling a single gate.* Thus, we believe this is a natural assumption for proving lower bounds about garbling individual boolean gates.

- The evaluation algorithm must use what we call **coordinated random oracle queries.** When evaluating a boolean gate on input (i, j), for each random oracle query, the evaluation algorithm can correctly predict which other input combinations — $(i, 1-j), (1-i, j), (1-i, 1-j)$ — would also make this query. We observe this property to be self-evidently true for all known garbling schemes.

Thus, our lower bound model is the closest yet to the "dream" lower bound described above, avoiding assumptions about the internal behavior of the garbling and evaluation algorithms (*i.e.*, no assumptions of linearity, etc). The restrictions only involve how the evaluation algorithm accesses the random oracle, and does not restrict the garbling algorithm at all.

We consider the above restrictions *potentially* limiting. Our proof also relies on additional properties of a garbling scheme, which we consider universal/fundamental, but list here for the sake of completeness:

- The pair of wire labels on every wire are either uncorrelated (as in classic textbook Yao garbling), or globally correlated as in free-XOR. In particular, we require the input labels and output labels of a gate to have the same correlation, thus ensuring that the garbled gates are composable and can be used for every gate in a circuit. This excludes from our model the constructions in [KKS16, BMR16, WM17], which garble an AND gate for λ bits, but are not composable.

- Gates are garbled from the inputs towards the outputs. That is, the garbling algorithm does not get to choose the input wire labels to a gate, but does get to choose the output label. This assumption seems fundamental to a garbling scheme designed for arbitrary circuits. In contrast, a garbling scheme specialized for *formulas* (*i.e.*, fan-out-1 circuits) would be free to garble from the output towards the inputs. An example of such a scheme is the privacy-free garbling scheme for formulas due to Kondi & Patra [KP17].

- Of course one can make a smaller garbled circuit by reducing the size of its wire labels; this would be equivalent to simply degrading the effective security parameter. By requiring that *wire labels are exactly λ bits* (and insisting in our security definition that they are pseudorandom), we ensure that the garbling scheme provides full λ-bit security. However, this requirement rules out garbling schemes whose input and output wire labels are different lengths, like the information-theoretic garbling of Kolesnikov [Kol05].

The security definition. We use a non-standard security definition that is slightly stronger than the usual one for garbling. In short, the security game reveals *all* wire labels to the evaulator at the end, but only after the adversary's access to the random oracle has been removed. In every garbling scheme that we know of, wire labels are used to determine random-oracle inputs. So it does no harm to reveal the entire set of wire labels if the random oracle is inaccessible.

This modified security definition provides us two benefits: (1) It can capture the standard garbling security properties (privacy, obliviousness, and authenticity), as well as correctness, in a single game. (2) The adversary sees all wire labels, making it easier to reason about entropy relationships among wire labels within the context of the game.

Ultimately, we prove our lower bound by considering *query-honest* adversaries in this security game. That is, the adversary only chooses to query the random oracle according to the honest evaluation algorithm. By considering a restricted class of adversaries, we only strengthen our result (even securing a garbled AND gate against a weaker adversary requires 1.5λ bits), while making the security definition more amenable to our information-theoretic techniques.

The lower bounds. We prove the following, for garbling schemes meeting the requirements listed above:

- A garbled AND-gate with free-XOR wire labels requires $1.5\lambda - \mathsf{negl}(\lambda)$ bits.

- A garbled AND-gate with uncorrelated wire labels requires $2\lambda - \mathsf{negl}(\lambda)$ bits.

- A garbled XOR-gate with uncorrelated wire labels requires $1\lambda - \mathsf{negl}(\lambda)$ bits.

Thus the free-XOR-compatible garbling scheme of Rosulek & Roy [RR21] is **optimal**, as is the non-free-XOR scheme of Gueron, Lindell, Nof, and Pinkas [GLNP15].

1.2 Overview of Our Techniques

We first give an equivalent characterization of our security game in terms of a collection of global *garbling joint distributions* over the relevant values: wire labels, garbled gate, and random oracle responses. There is one garbling distribution for each "flavor" of the security game (choice of gate, and choice of evaluator's input). An important part in this step is how we represent the random oracle responses in this distribution.

We call an oracle query *2-way* if it can be made by 2 different input combinations to the garbled gate. For example, if the input labels are (A_0, A_1) on one wire and (B_0, B_1) on the other, then an oracle query of the form $H(A_0)$ is 2-way because it can be made by the $(0, 0)$ and $(0, 1)$ input combinations, but not the others. Similarly, an oracle query of the form $H(A_1, B_0)$ is *1-way* because it can be made only by the (1,0) input combination.

Our global garbling distributions therefore encode the random oracle responses based on which input combinations make the queries. For example, one part of the encoding contains responses to 2-way oracle queries made by the (0,1) and (1,0) input combinations. In order for everything to work out, it is important that the evaluation algorithm makes *non-adaptive* and *coordinated* queries (defined above).

About n-way oracle queries. Characterizing oracle queries as n-way is essential to our technique. We show the following facts about oracle queries:

- Without loss of generality, a garbling scheme with *non-adaptive* oracle queries does not make 3-way queries. More precisely, any 3-way query is actually a 4-way query.
- Without loss of generality, all 1-way queries can be expressed in terms of 2-way queries. The intuition is the following: the "canonical" 1-way oracle query is $H(A_i, B_j)$, but the same effect can be achieved by making two 2-way queries $H(A_i) \oplus H(B_j)$. We express this intuition in terms of our global garbling distributions, re-encoding the 1-way oracle response information as a part of the 2-way random variables.
- If wire labels have free-XOR correlations, then an oracle query of the form $H(A_0 \oplus B_0)$ is a 2-way query shared by input combinations (0,0) and (1,1), because $A_0 \oplus B_0 = (A_0 \oplus \Delta) \oplus (B_0 \oplus \Delta) = A_1 \oplus B_1$. But if the scheme has uncorrelated (*i.e.*, non-free-XOR) wire labels, we show that there is no 2-way oracle query for input combinations $\{(0, 0), (1, 1)\}$ nor for $\{(0, 1), (1, 0)\}$.

By making all these observations, we not only distill the important differences between free-XOR and non-free-XOR garbling, but we reduce the number of random variables required to describe our global garbling distributions. The latter feature is important for the next step of our approach.

Information bounds. We now have a garbling distribution for each version of the security game. We show various information-theoretic bounds about these distributions. These bounds take the following forms:

- In some garbling distributions, certain values are independent by definition; some values can be computed as functions of other values. We can express such relationships in terms of entropy. For example, independently distributed values have zero mutual entropy. If X is computable from Y then X has zero conditional entropy conditioned on Y.

- We can use the fact that certain games are indistinguishable to translate bounds on one garbling distribution to a matching bound on another distribution. For example, if X and Y have high mutual entropy in one garbling distribution, then we can deduce that the same is true in another garbling distribution. While applying indistinguishability in this way, a negligible error term is added to the new bound.

Solver for Shannon bounds. All of the bounds derived above are *Shannon bounds.* That is, they can be expressed as simple linear inequalities of conditional mutual entropy expressions. Suppose that among all of the information bounds derived above, there are N possible mutual entropy expressions. Then one can characterize each probability distribution by a length-N vector, whose ith component is the value of the ith mutual entropy expression.

A Shannon bound therefore defines a half-space within the space of all probability distributions. The conjunction of many Shannon bounds defines a polytope. Thus, we can use *linear programming* techniques to identify extremal probability distributions within the polytope. Essentially, there is a linear program that minimizes the entropy (and hence size) of the garbled gate information, subject to the information bounds derived above (and the set of all axiomatic Shannon bounds).

There exist tools that support solving such Shannon-bound linear programs. We use one called `Citip`, by Gläßle [Glä15], adapted from an earlier program `Xitip`, by Pulikkoonattu, Perron, and Diggavi [PPD07]. We made significant modifications to Citip, so that it can support reasoning about multiple garbling distributions within a single linear program, and so that it can support the "slack" terms that arise when applying our indistinguishability bounds. Our changes are available at `https://github.com/ldr709/Citip`.

1.3 Other Related Work

The idea of deriving lower bounds on cryptographic constructions from information bounds is not new. The work of Csirmaz [Csi95] introduced the technique in the context of lower bounds on secret sharing schemes. Many other works have further explored the connection between information bounds and secret sharing — notably, Beimel and Orlov [BO09] explicitly consider non-Shannon bounds, while Padró and Vázquez [PV10] explicitly consider the use of a linear program solver, similar to our approach. We note that the process of expressing security requirements as entropy inequalities is far more straightforward for secret sharing than garbling.

2 Information Theoretic Preliminaries

2.1 Shannon Bounds

Definition 1. *We use $\Psi_P(X;Y \mid Z)$ to denote conditional mutual information between X and Y, given Z, when the underlying probability distribution is P. By definition:*

$$\Psi_P(X;Y \mid Z) = \mathop{\mathbb{E}}_{X,Y,Z \leftarrow P}\left[\log\left(\frac{P(X,Y \mid Z)}{P(X \mid Z)P(Y \mid Z)}\right)\right].$$

Note that $\Psi_P(X;Y \mid Z)$ is always symmetric between X and Y. We use the following shorthand notations:

$$\Psi_P(X) \stackrel{\text{def}}{=} \Psi_P(X;X \mid \bot) \qquad \text{the entropy of } X$$

$$\Psi_P(X \mid Z) \stackrel{\text{def}}{=} \Psi_P(X;X \mid Z) \qquad \text{the conditional entropy of } X \text{ given } Z$$

$$\Psi_P(X;Y) \stackrel{\text{def}}{=} \Psi_P(X;Y \mid \bot) \qquad \text{the mutual information between } X \text{ and } Y$$

Remark 2. *The traditional choice for the base of the entropy logarithm is 2, so that the units of entropy are* bits. *In our setting, it is more convenient to* ***measure entropy in units of λ-bit blocks.*** *Thus we use 2^λ as the base of the logarithm.*

So a uniform distribution over $\{0,1\}^\lambda$ has 1 unit of entropy. Looking ahead, our main result will conclude that a garbled AND-gate requires 1.5 units of entropy, thus requiring 1.5λ bits.

We use a number of textbook results of information theory. See, e.g., [Cov99].

Proposition 3. *Conditional information is the amount of information learned by seeing X, when Z is already known:* $\Psi(X \mid Z) = \Psi(X, Z) - \Psi(Z)$.

Proposition 4. *The mutual information of X and Y is how much less uncertain X becomes after seeing Y:* $\Psi(X;Y \mid Z) = \Psi(X \mid Z) - \Psi(X \mid Y, Z)$.

Proposition 5. *Information cannot be negative:* $\Psi(X;Y \mid Z) \geq 0$.

Many (in)equalities can be derived from these rules. We use the following:

Corollary 6.

$$\begin{aligned}
\Psi(X;Y \mid Z) &= \Psi(X \mid Z) + \Psi(Y \mid Z) - \Psi(X, Y \mid Z) \\
\Psi(X;Y \mid Z_1, Z_2) &= \Psi(X, Z_1 \mid Z_2) + \Psi(Y, Z_1 \mid Z_2) - \Psi(X, Y, Z_1 \mid Z_2) - \Psi(Z_1 \mid Z_2) \\
\Psi(X, W;Y, W \mid Z) &= \Psi(W \mid Z) + \Psi(X;Y \mid W, Z) && \textit{generalized chain rule} \\
\Psi(X;Y_1, Y_2 \mid Z) &= \Psi(X;Y_1 \mid Z) + \Psi(X;Y_2 \mid Y_1, Z) && \textit{one-sided chain rule} \\
\Psi(X \mid Y, Z) &\leq \Psi(X \mid Z) && \textit{learning reduces entropy} \\
\Psi(X, Y \mid Z) &\geq \Psi(X \mid Z) && \textit{more variables gives more entropy} \\
\Psi(X;Y, Y' \mid Z) &\geq \Psi(X;Y \mid Z) && \textit{same, but for mutual information} \\
\Psi(X;Y \mid Z) &\geq \Psi(X; f(Y) | Z) && \textit{data-processing inequality}
\end{aligned}$$

Definition 7. *Any inequality on Ψ that can be derived from (only) the rules above is called a* ***Shannon Inequality****.*

2.2 Bounding Entropies and Probabilities

So far, we have just talked about measuring entropy and information, and relating different entropies to each other. However, we will also need to relate these entropies to correctness and security properties of a garbled circuit. Here we present several bounds relating entropy to probabilistic properties.

First, we have that if Y is a deterministic function of X, then Y has no entropy when conditioned on X: $\Psi_P(X \mid Y) = 0$. More generally, if Y is often a deterministic function of X, then Fano's inequality upper bounds the entropy of Y conditioned on X.

Lemma 8 (Fano's inequality)**.** *Let X and Y be random variables and f be a function. If $\Pr[Y \neq f(X)] \leq \epsilon$ and the support of Y has cardinality N then $\Psi(Y \mid X) \leq \epsilon \log(N-1) + \phi(\epsilon)$, where $\phi(\epsilon) = -\epsilon \log \epsilon - (1-\epsilon)\log(1-\epsilon)$.*

Sketch. The intuition that is Y can be encoded by sending X, then a bit $(Y \neq f(X))$, and if this bit is 1 then also sending which one of the $N-1$ remaining possibilities for Y is correct. See, e.g., [Cov99] for a full proof. □

Conversely, if $\Psi_P(Y \mid X) = 0$ then Y is a deterministic function of X. The approximate version of this is based the inequality between entropy and min-entropy.

Lemma 9. *For any random variables X and Y, there exists a function $f(X)$ such that* $\Pr[Y \neq f(X)] \leq 1 - 2^{-\lambda \Psi_P(Y|X)} \leq \lambda \ln(2)\, \Psi_P(Y \mid X)$.

Next, if Y is sampled freshly as a randomized function of X, then conditioned on X, the Y is independently random of any variable Z that was chosen before it. That is, $\Psi(Y; Z \mid X) = 0$.

Finally, many security properties are defined in terms of distributions that must be indistinguishable. Obviously, if P and Q are perfectly indistinguishable, it means that the entropies (and mutual informations, etc.) of any subset of variables are the same between the two. However, garbling schemes are not required to be perfectly secure, so P and Q won't be perfectly indistinguishable. We need a relationship between the statistical distance between distributions, and the difference in their entropies.

Lemma 10. *Let P and Q be two distributions over random variables X, Y with total variation distance $\frac{1}{2}\|P - Q\|_1 = \epsilon$. Let $\tilde{\phi}(\epsilon) = (1+\epsilon)\log(1+\epsilon) - \epsilon \log \epsilon$. If the support of X is a finite set with cardinality N, then*

$$|\Psi_P(X \mid Y) - \Psi_Q(X \mid Y)| \leq \epsilon \log(N) + \tilde{\phi}(\epsilon),$$

and similarly for distributions over three random variables X, Y, Z:

$$|\Psi_P(X; Y \mid Z) - \Psi_Q(X; Y \mid Z)| \leq \epsilon \log(N) + 2\tilde{\phi}(\epsilon).$$

Proof. The first bound was proven by Winter [Win16, Lemma 2], whose technique Shirokov adapted to prove the second bound [Shi17, Corollary 2]. These bounds were stated for quantum-classical densities, rather than classical distributions, meaning that Y was a classical state that is correlated to a quantum state representing (X, Y).[1] Any classical distribution can be turned into a quantum density matrix, by making a diagonal matrix out of the probabilities. Consequently, these quantum bounds imply their corresponding classical bounds. □

Remark 11. *Note that these bounds are nearly tight. Let the distributions be* $P(X,Y) = \begin{cases} 1 & \text{if } X = 0 \\ 0 & \text{otherwise} \end{cases}$ *and* $Q(X,Y) = \begin{cases} 1-\epsilon & \text{if } X = 0 \\ \frac{\epsilon}{N-1} & \text{otherwise} \end{cases}$*, supported on the integers $\{0, \ldots, N-1\}$. Then $\frac{1}{2}\|P - Q\|_0 = \epsilon$, $\Psi_P(X \mid Y) = 0$, and $\Psi_Q(X \mid Y) = \epsilon \log(N-1) - \epsilon \log(\epsilon) - (1-\epsilon)\log(1-\epsilon)$, which differs from the bound by $O(\epsilon/N + \epsilon^2)$. A similar distribution on (X, Y, Z), defined by setting $X = Y$, shows that the bound on conditional mutual information is also nearly tight.*

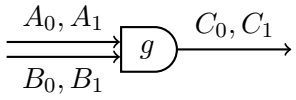
3 Garbling Schemes

3.1 Syntax

Our focus is on garbling for individual boolean gates, so our definitions are specialized for this case.

Roughly speaking, a garbling scheme associates to each wire in a circuit two **labels**, each strings of length λ. The two labels represent false & true plaintext values on the wire. Suppose an evaluator holds the labels that correspond to plaintext values i and j on the input wires of a gate. The goal of a garbled circuit is to provide a way for the evaluator to learn the label that encodes $g(i, j)$ on the output wire.

[1] Strangely, the classical bounds given in the literature seem to be less tight.



Privacy vs gate-hiding: With the point-and-permute optimization, every wire label carries a **color bit**, a single bit of information that is visible to the evaulator. The two labels on a single wire have opposite color bits, and the choice of color bits is independent of the plaintext values they encode.

Suppose the subscripts in A_0, A_1, B_0, B_1 denote the color bits. Then the garbler will know two secret *permute bits* π_A and π_B, such that $A_{i \oplus \pi_A}$ encodes plaintext value i and $B_{j \oplus \pi_B}$ encodes plaintext value j. The traditional view of garbling holds that the choice of gate g is public, but the association between plaintext values and wire labels is a secret, known only to the garbler.

An alternative interpretation is to view the gate's functionality as a computation on the public *color bits.* In this interpretation, we imagine that color bits are the plaintext values of the computation. The garbler then chooses a secret function of the form $(i, j) \mapsto g(i \oplus \pi_A, j \oplus \pi_B) \oplus \pi_C$ which will be computed on these plaintext values.

The traditional view of *private plaintext values and public gate function* is therefore equivalent to this new view of *public plaintext values and private choice of gate function.* In this work we take the latter interpretation. Thus, our security definitions will be in terms of gate-hiding, and both garbler and evaluator have the same "dialect" for the wire labels.

Under this perspective, garbling and AND-gate corresponds to garbling the following class of gates in a gate-hiding manner:

$$\mathcal{G}_{\mathsf{and}} = \{(i, j) \mapsto (i \oplus \pi_A) \wedge (j \oplus \pi_B) \oplus \pi_C \mid \pi_A, \pi_B, \pi_C \in \{0, 1\}\}$$

Syntax: Our definition below includes the point-and-permute optimization "for free," meaning that the evaluation algorithm is explicitly given the subscripts / color bits of wire labels. This only strengthens our lower bounds, since a garbling scheme can choose to ignore this extra information. The garbling algorithm also includes a choice of gate function.

Definition 12. *A **gate garbling scheme** for a class $\mathcal{G}$ of boolean gates consists of the following algorithms:*

- $\mathsf{Gb}^H(g, A_0, A_1, B_0, B_1) \to (G, C_0, C_1)$*: Given a choice of gate $g \in \mathcal{G}$, and input wire labels, produces a garbled gate and two output labels.*
- $\mathsf{Ev}^H(G, A_i, B_j, i, j) \to C$*: Given a garbled gate, and input labels corresponding to input combination (i, j), produces an output label.*

Correctness refers to the fact that evaluating such a gate G on A_i and B_j results in an output $C_{g(i,j)}$. This correctness property will be defined formally in our security definition below.

Wire labels. Our goal is a fine-grained lower bound, resolving optimal constant factors in the size of garbled circuits. To provide a fair playing field on which to compare garbling schemes, we require that wire labels are strings of λ bits.

Inputs-to-outputs. Finally, we note that our definition assumes that garbling happens from inputs to outputs. The garbling procedure is given the input wire labels; it cannot choose them. It can, however, choose the output labels (and of course the garbled gate).

3.2 Restrictions on Oracle Queries

Our results hold for Ev that makes non-adaptive oracle queries:

Definition 13. *A gate garbling scheme* $(\mathsf{Gb}, \mathsf{Ev})$ *has* ***non-adaptive queries*** *if the* Ev *algorithm makes only non-adaptive queries to its oracle. I.e., it can make all queries to its oracle in a single, parallel batch. (We make no restrictions on the* Gb *algorithm.)*

The **"(i,j) evaluator"** refers to an execution of the Ev algorithm on inputs A_i, B_j (along with the color bits i and j). We require that whenever Ev makes an oracle query, it must know which other evaluators will also make the same query. More formally:

Definition 14. *A gate garbling scheme* $(\mathsf{Gb}, \mathsf{Ev})$ *has* ***coordinated queries*** *if the following conditions hold:*

- *Every oracle query made by* Ev *has the form* $H(S, X)$ *where* $S \subseteq \{0,1\}^2$*. Think of* S *as a subset of evaluators* $(i, j$ *pairs).*
- *For arbitrary* $A_0 \neq A_1$*,* $B_0 \neq B_1$*, and* $g \in \mathcal{G}$*, the following procedure never aborts:*

$(G, \cdot, \cdot) \leftarrow \mathsf{Gb}^H(g, A_0, A_1, B_0, B_1)$
for $(i,j) \in \{0,1\}^2$:
  $Q_{i,j} := \{\text{all oracle queries made by } \mathsf{Ev}^H(G, A_i, B_j, i, j)\}$
$Q_{all} := Q_{0,0} \cup Q_{0,1} \cup Q_{1,0} \cup Q_{1,1}$

for each $S \subseteq \{0,1\}^2$:
  if $\left\{(S,X) \,\middle|\, \exists X : (S,X) \in Q_{all}\right\} \neq \left(\bigcap_{(i,j)\in S} Q_{i,j}\right) \cap \left(\bigcap_{(i,j)\notin S} \overline{Q_{i,j}}\right)$: abort

In the final line, the left-hand side is the set of all queries (made by any evaluator) that are labeled with S*. The right-hand side is the set of all queries actually made by (only) the evaluators named in* S*. The two sets must be identical. Intuitively, each query* $H(S, X)$ *correctly identifies the set* S *of evaluators that will make this query.*

We say that an oracle query $H(S, X)$ *is* k***-way*** *if* $|S| = k$*.*

Examples. In classical Yao garbling (augmented by the point-and-permute technique [BMR90]), the garbled gate consists of 4 ciphertexts of the form $H(A_i, B_j) \oplus C_k$. This is a one-time encryption of C_k using A_i and B_j as the key. The oracle query $H(A_i, B_j)$ is a 1-way query because only the evaluator (i, j) can make it. The scheme could be modified to meet our definition of coordinated queries by changing the oracle queries to be $H(\{(i,j)\}, A_i, B_j)$.

In the half-gates garbling scheme of Zahur, Rosulek, and Evans [ZRE15], the evaluator on input (A_i, B_j) makes oracle queries of the form $H(A_i)$ and $H(B_j)$. These are 2-way queries, because (for example) $H(A_0)$ can be made by the (0,0) and (0,1) evaluators. The scheme can be trivially modified to meet our definition, by renaming $H(A_i)$ queries to $H(\{(i,0),(i,1)\}, A_i)$ and $H(B_j)$ queries to $H(\{(0,j),(1,j)\}, B_j)$.

In the three-halves scheme of Rosulek and Roy [RR21], the evaluator makes queries of the form $H(A_i \oplus B_j)$. When the wire labels have the free-XOR correlation, these are two-way queries between the (i,j) and $(1-i, 1-j)$ evaluators, because $A_i \oplus B_j = A_{1-i} \oplus B_{1-j}$.

3.3 Free-XOR vs Plain Garbling

Different garbling schemes use different distributions on wire labels. In textbook garbling, the labels A_0, A_1 on a single wire are uncorrelated. But in the free-XOR method, these wires satisfy $A_0 \oplus A_1 = \Delta$, where Δ is a secret constant, global to *all* wires in the circuit.

In this work we consider these two common cases of wire label correlation in a unified way. We define the following functions, with the following interpretation. If $W_1, \ldots, W_n$ are wire labels (think one for each wire), then $\mathsf{CompLbl}(W_1, \ldots, W_n)$ samples complementary wire labels — *i.e.*, on each wire, the *other* label, encoding the complementary bit.

$\underline{\mathsf{CompLbl}_{\mathsf{plain}}(W_1, W_2, \ldots, W_n)}$:
for $i = 1$ to n:
  $W_i' \leftarrow \{0,1\}^\lambda$
return $(W_1', W_2', \ldots, W_n')$

$\underline{\mathsf{CompLbl}_{\mathsf{free\text{-}xor}}(W_1, W_2, \ldots, W_n)}$:
$\Delta \leftarrow \{0,1\}^\lambda$
for $i = 1$ to n:
  $W_i' := W_i \oplus \Delta$
return $(W_1', W_2', \ldots, W_n')$

Remark 15. *The existence of a* CompLbl *function implies that wire label correlations are* symmetric. *The* CompLbl *function samples a label's partner, without knowing whether it encodes true or false. Other forms of wire label correlation do not enjoy this property. For example, the λ-bit AND-gates of [KKS16, BMR16, WM17] use wire labels of the form $(W, W + \Delta)$ where the addition does not have characteristic 2. So the partner of a wire label W is either $W + \Delta$ or $W - \Delta$, which are different values, depending on whether W encodes false or true.*

3.4 Security Definition

We require a somewhat non-standard, simulation-based security for garbling.

Definition 16. *Define the following games, played against a two-stage adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$:*

$\underline{\textsc{real}(g, i, j)}$:
$A_0, B_0 \leftarrow \{0,1\}^\lambda$
$(A_1, B_1) \leftarrow \mathsf{CompLbl}(A_0, B_0)$
$(G, C_0, C_1) \leftarrow \mathsf{Gb}^H(g, A_0, A_1, B_0, B_1)$
$\sigma \leftarrow \mathcal{A}_1^H(G, A_i, B_j, i, j)$
$k = g(i, j)$
return $\mathcal{A}_2(\sigma, A_{1-i}, B_{1-j}, C_k, C_{1-k})$

$\underline{\textsc{ideal}(i, j, k)}$:
$A_i, B_j \leftarrow \{0,1\}^\lambda$
$(G, H) \leftarrow \mathcal{S}(A_i, B_j, i, j)$
$\sigma \leftarrow \mathcal{A}_1^H(G, A_i, B_j, i, j)$
$C_k := \mathsf{Ev}^H(G, A_i, B_j, i, j)$
$(A_{1-i}, B_{1-j}, C_{1-k}) \leftarrow \mathsf{CompLbl}(A_i, B_j, C_k)$
return $\mathcal{A}_2(\sigma, A_{1-i}, B_{1-j}, C_k, C_{1-k})$

We say that the garbling scheme is ***strongly secure*** *with respect to a class of gates $\mathcal{G}$ if there exists a simulator $\mathcal{S}$ such that for all $i, j \in \{0,1\}$ and $g \in \mathcal{G}$, and all computationally unbounded $\mathcal{A}$ that make polynomially many oracle queries (i.e., Minicrypt adversaries), the games $\textsc{real}(g, i, j)$ and $\textsc{ideal}(i, j, g(i, j))$ are indistinguishable.*

This definition requires some explanation.

- $\textsc{real}(g, i, j)$ is a standard scenario for garbled circuits. The gate g is garbled honestly, with suitably correlated input labels. The adversary sees the garbled gate and the input labels encoding (i, j). The only difference is that, *after revoking the adversary's access to the random oracle,* the complementary wire labels are revealed.

- In IDEAL(i, j, k), a simulator simulates the adversary's view, but without knowledge of the gate g. (Recall that in our convention, the only secret hidden by the garbled gate is the choice of gate.) The simulator must simulate the garbled gate and random oracle. The adversary sees *visible* labels for the input wires (A_i and B_j), and, implicitly, sees a visible label C_k on the output wire. The complementary wire labels (revealed at the end of the game) are required to be correlated to these visible wire labels in the expected way.

To the best of our knowledge, all known garbling schemes satisfy this stronger security property. In all known schemes, wire labels are used as inputs to the random oracle. Security rests on the adversary's inability to query the oracle at the complementary wire labels. It therefore does no harm to reveal those wire labels after the adversary has no chance to query the oracle.

Implications: This definition implies all typical security and correctness properties of a garbling scheme (*i.e.*, the standard definitions from [BHR12]):

- *Correctness:* In the last step of REAL, the adversary is given $C_{g(i,j)}$ computed by Gb, but in IDEAL it is given C_k computed by Ev. These values must be indistinguishable.
- *Privacy & obliviousness:* The garbling scheme hides the choice of g and the gate's plaintext output value, because the IDEAL game does not have access to these values.
- *Authenticity:* The standard definition of authenticity says, roughly, that given labels A_i and B_j, it is possible to compute $C_{g(i,j)}$ but difficult to compute the complementary label $C_{1-g(i,j)}$. Our definition expresses this property in more of a *real-vs-random* paradigm. The adversary is given all complementary wire labels at the end of the game. The *real* complementary labels are indistinguishable from truly random ones (subject to their being suitably correlated according to $\mathsf{CompLbl}$).

We also remark that our security definition demands that the gate's output labels share the same correlation as the input labels. This is evident by the call to $\mathsf{CompLbl}$ in IDEAL, made on both the input and output labels. Without this requirement, a gate-garbling scheme would not be composable with itself (like the λ-bit AND-gate constructions of [KKS16, BMR16, WM17]).

4 An Information-Theoretic View of Garbling

The adversary's view in the security game is a distribution over various components: wire labels, garbled gate information, and random oracle outputs. We will establish various information inequalities about these components of the distributions; then, we will use an automated theorem prover for Shannon inequalities to conclude a lower bound on the size of the garbled gate. The cost of the automated prover scales exponentially with the number of random variables, so it is important to express the adversary's view in terms of the fewest random variables.

The biggest challenge is our treatment of the random oracle. In particular, we do not want to introduce a separate random variable for each random oracle response. Instead, we use the following approach.

First, we restrict our focus to *query-honest* adversaries, who make only the oracle queries needed to run Ev honestly. We emphasize that restricting adversaries only strengthens our lower bound (achieving security, even against a limited class of adversaries, still requires a certain size of garbled circuit).

Second, we take advantage of the fact that the garbling scheme has *coordinated queries* (Definition 14). Thus, each oracle query is labeled with the set of evaluators who will make it. If two oracle queries are made by the same set of evaluatories, their responses can be encoded into the same random variable for the purposes of our information bounds. In other words, for each subset of evaluators $S \subseteq \{0,1\}^2$, we (informally) define a random variable H_S encoding the oracle responses made by those evaluators.

$\underline{\text{GDIST}(g)}$:
  $A_0, B_0 \leftarrow \{0,1\}^\lambda$
  $(A_1, B_1) \leftarrow \mathsf{CompLbl}(A_0, A_1)$
  $H \leftarrow$ random oracle
  $(G, C_0, C_1) \leftarrow \mathsf{Gb}^H(g, A_0, A_1, B_0, B_1)$
  for $(i,j) \in \{0,1\}^2$:
    $Q_{i,j} = \{$all oracle queries made by $\mathsf{Ev}^H(G, A_i, B_j, i, j)\}$
  $Q_{all} = \bigcup_{i,j} Q_{i,j}$

  for all $q \in Q_{all}$, in lexicographic order:
    write $q = (S, X)$, since scheme has coordinated queries
    append oracle response $H(S, X)$ to the list H_S (initially empty)
  output $(A_0, A_1, B_0, B_1, G, C_0, C_1, \{H_S\}_S)$

// only outputs H_S that are relevant for evaluator (i,j):
$\underline{\text{GDIST}(g, i, j)}$:
  $(A_0, A_1, B_0, B_1, G, C_0, C_1, \{H_S\}_S) \leftarrow \text{GDIST}(g)$
  output $(A_0, A_1, B_0, B_1, G, C_{g(i,j)}, C_{1-g(i,j)}, \{H_S \mid S \ni (i,j)\})$

It is assumed that $\{H_S\}_S$ is encoded as the *sequence* $H_{S_1}, H_{S_2}, \ldots$, where $S_1, S_2, \ldots$ is a canonical ordering of the possible values of S.

$\underline{\text{SIMDIST}(i, j, k)}$:
  $A_i, B_j \leftarrow \{0,1\}^\lambda$
  $(G, H) \leftarrow \mathcal{S}(A_i, B_j, i, j)$ *// H is the entire (simulated) random oracle*
  $C_k = \mathsf{Ev}^H(G, A_i, B_j, i, j)$

  for every oracle query q made by Ev^H above, in lexicographic order:
    write $q = (S, X)$, since scheme has coordinated queries
    append oracle response $H(S, X)$ to the list H_S (initially empty)

  $(A_{1-i}, B_{1-j}, C_{1-k}) \leftarrow \mathsf{CompLbl}(A_i, B_j, C_k)$
  output $(A_0, A_1, B_0, B_1, G, C_k, C_{1-k}, \{H_S \mid S \ni (i,j)\})$

Lemma 17. *A garbling scheme with coordinated queries has security against a query-honest adversary* ***if and only if*** *for all $i, j \in \{0,1\}$ and $g \in \mathcal{G}$, the distributions* $\text{GDIST}(g, i, j)$ *and* $\text{SIMDIST}(i, j, g(i,j))$ *are indistinguishable.*

Proof. ($\Rightarrow$) Consider the following query-honest adversary in the garbling security game.

$\mathcal{A}_1^H(A_i, B_j, i, j, G)$:
  run $\mathsf{Ev}^H(G, A_i, B_j, i, j)$
  for every oracle query q made by Ev^H, in lexicographic order:
    write $q = (S, X)$
    append the response $H(S, X)$ to the list H_S, initially empty
  output $\sigma = (A_i, B_j, G, \{H_S \mid S \ni (i, j)\})$

$\mathcal{A}_2(\sigma, A_{1-i}, B_{1-j}, C_k, C_{1-k})$:
  parse σ as $(A_i, B_j, G, \mathcal{H})$
  return $(A_0, A_1, B_0, B_1, G, C_k, C_{1-k}, \mathcal{H})$

This adversary produces output which is identically distributed to $\text{GDIST}(g, i, j)$ when in the $\text{REAL}(g, i, j)$ game, and identically distributed to $\text{SIMDIST}(i, j, k)$ when in the $\text{IDEAL}(i, j, k)$ game.

($\Leftarrow$) For any query-honest two-stage adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, consider the following reduction algorithm:

$\text{GAME}_{\mathcal{D}}(g, i, j)$:
  $(A_0, A_1, B_0, B_1, G, C_k, C_{1-k}, \{H_S\}_S) \leftarrow \mathcal{D}$
  run $\mathsf{Ev}^?(A_i, B_j, i, j, G)$ until it makes its batch of oracle queries

  for all queries q made by $\mathsf{Ev}^?$ above, in lexicographic order:
    write $q = (S, X)$
    remove the next item from H_S and call it R
    program an oracle (initially empty) at $H(S, X) = R$

  $\sigma = \mathcal{A}_1^H(A_i, B_j, i, j, G)$
  return $\mathcal{A}_2(\sigma, A_{1-i}, B_{1-j}, C_k, C_{1-k})$

Note that because $\mathcal{A}$ is query-honest, it may only query its oracle H on the points actually programmed by the game. When the distribution $\mathcal{D}$ is $\text{GDIST}(g, i, j)$, then the output of this game is identically distributed to $\text{REAL}(g, i, j)$. When $\mathcal{D}$ is $\text{SIMDIST}(i, j, k)$, then the output is identically distributed to $\text{IDEAL}(i, j, k)$.[2] □

Remark 18. $\text{SIMDIST}(i, j, k)$ *does not use the value* k *— i.e., we may as well rename* (C_k, C_{1-k}) *to* (C, C')*. Thus, we may simply write* $\text{SIMDIST}(i, j)$*.*

4.1 Optimization: Removing 1-way Queries

Our garbling distributions include a random variable H_S for every $S \subseteq \{0, 1\}^2$. It will be useful to minimize the number of such variables, since we eventually use a linear program whose size is exponential in the number of variables. In this section, we show how to eliminate the random variables that represent 1-way oracle queries (corresponding to $|S| = 1$).

The intuition is that a 1-way query of the form $H(A_i, B_j)$ can be replaced with $H(A_i) \oplus H(B_j)$, a combination of two 2-way queries. However, it is not easy to rewrite a completely *arbitrary* garbling scheme to replace its 2-way queries by 1-way queries in this manner. The problem is that

[2] This is the step where we use the coordinated queries property of the garbling scheme. GAME can observe the oracle queries made by $\mathsf{Ev}^?(\cdots, i, j)$, but unless they are suitably labeled, it has no way to know which other evaluators make the same queries, and thus which H_S list contains them. This is also one of several steps where the non-adaptivity of Ev's oracle queries is important.

there can be many 1-way queries and many existing 2-way queries, and the query-rewriting must not introduce conflicts.

Instead, we leave the garbling scheme as-is, with its 1-way queries, but encode the oracle responses across the H_S random variables for 2-way queries. Taking inspiration from the previous example, we *secret-share* the result of a 1-way query $H(A_i, B_j)$ between the random variables for $H(A_i)$-style 2-way queries and $H(B_j)$-style queries.

More formally, we modify GDist as follows:

GDist(g):

⋮ *// previous lines same as before*

for all $q \in Q_{all}$, in lexicographic order:
  write $q = (S, X)$, since scheme has coordinated queries
  append oracle response $H(S, X)$ to the list H_S (initially empty)
  if $|S| = 1$:
    write $S = \{(i, j)\}$
    // secret-share $H(S, X)$ between two 2-way variables
    $R \leftarrow \{0,1\}^\lambda$
    append R to the list $K_{(1-i,j)\to(i,j)}$ (initially empty)
    append $R \oplus H(S, X)$ to the list $K_{(i,1-j)\to(i,j)}$ (initially empty)
  else $|S| \neq 1$: *// same as before*
    append oracle response $H(S, X)$ to the list H_S (initially empty)

// include secret shares in 2-way H_S random variables:
for each $S \subseteq \{0,1\}^2$ with $|S| = 2$:
  write $S = \{(i, j), (i', j')\}$ with $2i + j \leq 2i' + j'$
  $H_S = (H_S, K_{(i,j)\to(i',j')}, K_{(i',j')\to(i,j)})$

output $(A_0, A_1, B_0, B_1, G, C_0, C_1, \{H_S\}_S)$

Intuitively, a value that is meant for H_S, with $|S| = 1$ is instead secret-shared, with the shares being placed in lists $K_{(1-i,j)\to(i,j)}$ and $K_{(i,1-j)\to(i,j)}$. Then the two-way random variables H_S, $|S| = 2$, are augmented to include these lists of secret shares.

An analogous change can be made in SimDist. The simulator provides a programmed random oracle H, then, for each oracle query (S, X) made by Ev^H, SimDist places the relevant oracle output into the H_S random variables. Now, instead, when $|S| = 1$, SimDist can secret-share the oracle output and place it in the appropriate 2-way H_S random variables as in GDist.

Lemma 17 still holds because the reduction algorithms in that proof can perform the necessary encoding/decoding of 1-way query results. The only case that requires care is that the evaluator (i, j) sees $H_{(i,j),(i',j')}$, which includes secret shares $K_{(i,j)\to(i',j')}$ and $K_{(i',j')\to(i,j)}$. The former are uniformly random from this evaluator's point of view, and can be simulated as such. The latter are secret shares of 1-way queries made by this evaluator (i, j), and the reduction algorithm has access to these 1-way queries and can secret-share the responses.

5 Information Bounds

In this section we prove various information bounds on the garbling distributions defined in the previous section. We use the following notation:

- $\Psi_{i,j}$ refers to conditional mutual entropy with respect to the $\textsc{SimDist}(i,j)$ distribution.
- Ψ_g refers to conditional mutual entropy with respect to the $\textsc{GDist}(g)$ distribution. Recall that $\textsc{GDist}(g,i,j)$ is computed as a function of the more general $\textsc{GDist}(g)$.

5.1 Simple Bounds

As discussed previously, our security definition implies correctness as well as standard security notions for garbling schemes. We rephrase some of these as information bounds, below.

We introduce the following shorthand notation:

$$H_{ij} \stackrel{\text{def}}{=} \{H_S \mid S \ni (i,j)\}.$$

Thus, H_{ij} refers to the random oracle responses that evaluator (i,j) is allowed to see. We sometimes refer to the collection (A_i, B_j, G, H_{ij}) as the **view** of evaluator (i,j). The following are immediate:

1. *Correctness:* $\Psi_{i,j}(C_{g(i,j)} \mid H_{ij}, A_i, B_j, G) = 0$.
   In $\textsc{SimDist}$, the output label is a function of the evaluator's view.
2. *Output authenticity:* $\Psi_{i,j}(C_{1-k}; H_{ij}, G \mid A_i, B_j, C_k) = 0$.
   In $\textsc{SimDist}$, the complementary wire labels are sampled conditioned only on A_i, B_j, C_k, so knowledge of H_{ij}, G gives no additional information about these complementary labels.
3. *Input authenticity:* $\Psi_{i,j}(A_{1-i}, B_{1-j}; H_{ij}, G \mid A_i, B_j) = 0$.
   By the same reasoning as above, knowledge of H_{ij} and G gives no additional information about the complementary *input* wire labels A_{1-i} and B_{1-j}.

5.2 Translating Bounds Between Distributions

When two distributions are indistinguishable, we may translate entropy bounds between them.

Lemma 19 (Between real & ideal)**.** *Suppose an information bound $\Psi_{i,j}(\alpha; \beta \mid \gamma) \le t$ holds, and α, β, γ are functions of the following values:*

$$A_0, A_1, B_0, B_1, C_0, C_1, G, H_{ij}. \tag{$\star$}$$

Then, assuming the garbling scheme is secure with simulation error ϵ, we also have

$$\Psi_g(\alpha; \beta \mid \gamma) \le t + \epsilon \log(N) + 2\tilde{\phi}(\epsilon),$$

where N is the cardinality of the support of α, and the log *uses the same base as the entropy* log*. In other words, the bound from ideal world (i,j) holds (approximately) for the real world g as well.*

Proof. The main idea is that the values in ($\star$) — and hence α, β, and γ — can be computed by the adversary in both $\textsc{ideal}(i, j, g(i,j))$ and $\textsc{real}(g, i, j)$ games. Thus the distribution of α, β, γ is identical in the two games. The lemma follows by applying Lemma 10. □

Lemma 20 (Between ideal & ideal)**.** *Suppose an information bound $\Psi_{i,j}(\alpha; \beta \mid \gamma) \le t$ holds, and α, β, γ are functions of the following values:*

$$A_0, A_1, B_0, B_1, C_0, C_1, G, H_{\{(i,j),(i',j')\}}.$$

Then, assuming the garbling scheme is secure with simulation error ϵ, we also have

$$\Psi_{i',j'}(\alpha;\beta \mid \gamma) \leq t + 2\epsilon \log(N) + 2\tilde{\phi}(2\epsilon),$$

where N is the cardinality of the support of α, and the log *uses the same base as the entropy* log*. In other words, the bound from ideal world (i,j) holds (approximately) for ideal world (i',j') as well.*

Proof. The values listed in this lemma can be computed by the adversary in all of the following games:

$$\textsc{ideal}(i,j,g(i,j)), \quad \textsc{real}(g,i,j), \quad \textsc{real}(g,i',j'), \quad \textsc{ideal}(i',j',g(i',j'))$$

The first two games are indistinguishable, as are the last two distributions, by the security of the garbling scheme. In the REAL games, the distribution of the values in ($\star$) does not depend on the choice of i,j. Thus the distribution of α,β,γ is identical in the two real games. By transitivity, $\textsc{ideal}(i,j,g(i,j))$ and $\textsc{ideal}(i',j',g(i',j'))$ induce indistinguishable distributions over (α,β,γ), with advantage 2ϵ. The lemma follows by applying Lemma 10. □

5.3 Almost-Independence of Random Oracle Responses

Distinct random oracle outputs are independently distributed; this is the defining characteristic of a random oracle. In our garbling distribution, random oracle responses are partitioned among the H_S random variables. Being a partition, these random variables are intuitively independent. However, this is not the case because the garbling algorithm has the freedom to choose which oracle queries are represented in H_S.

As an extreme example, suppose the garbling algorithm exhaustively queries the random oracle until it finds a value G leading to a collision $H(A_0,G) = H(A_1,G) = Z$. It then includes G in the garbled gate, and all evaluators can query $H(A_i,G)$, but via two distinct 2-way queries. Now two different H_S random variables will contain identical Z; they will not be independent.

However, this example introduces dependencies among H_S random variables at the cost of adding more information (G) to the garbled gate. We can show this to be true in general. Specifically,

Lemma 21. *For all g, we have $\Psi_g(\{H_S\}_S, G \mid A_0, A_1, B_0, B_1) \geq \sum_S \Psi_g(H_S)$. Equivalently,*

$$\Psi_g(\{H_S\}_S, G, A_0, A_1, B_0, B_1) \geq \sum_S \Psi_g(H_S) + \Psi_g(A_0, A_1, B_0, B_1)$$

Proof. The main idea is that if this inequality were violated, we could compress a random oracle below its entropy, which would be a contradiction. To begin with, we consider the GDIST which includes 1-way queries.

Suppose H is a truly random oracle, whose domain is a finite set of cardinality N, and whose outputs are strings of length λ. Recall that we measure entropy in units of λ-bit *blocks,* not *bits.* Thus, each random oracle output has entropy 1, and the random oracle itself has entropy N.

The input labels A_0, A_1, B_0, B_1 are independent of H, by construction. Thus,

$$\begin{aligned}\Psi_g(H, A_0, A_1, B_0, B_1) &= \Psi_g(H) + \Psi_g(A_0, A_1, B_0, B_1)\\ &= N + \Psi_g(A_0, A_1, B_0, B_1)\end{aligned}$$

Define $\overline{H}$ as H but with the values in H_S removed. More formally, for every random oracle query q *not* made by any evaluator (in GDIST), in lexicographic order, add $H(q)$ to the list $\overline{H}$. So $\overline{H}$ is like a punctured truth table of H, where the *locations* of the punctures are not provided.

We claim that H can be computed given $\{H_S\}_S$, G, and input labels A_0, A_1, B_0, B_1, as follows: Run every evaluator combination to identify their oracle queries. These oracle queries are the locations of the punctures in $\overline{H}$, and the punctures can be filled by taking entries from H_S, in lexicographic order of the query/puncture locations. Thus,

$$\begin{aligned} N + \Psi_g(A_0, A_1, B_0, B_1) &= \Psi_g(H, A_0, A_1, B_0, B_1) \\ &\leq \Psi_g(\overline{H}, G, \{H_S\}_S, A_0, A_1, B_0, B_1) \end{aligned}$$

Joint entropy is sub-additive, thus:

$$N + \Psi_g(A_0, A_1, B_0, B_1) \leq \Psi_g(\overline{H}) + \Psi_g(G, \{H_S\}_S, A_0, A_1, B_0, B_1)$$

Each H_S is a list of random oracle responses, each with 1 unit of entropy. Suppose H_S is a list of n_S entries, so that $\Psi_g(H_S) \leq n_S$. Applying the inequality for every S gives:

$$\begin{aligned} (N - \sum_S n_S) + \sum_S \Psi_g(H_S) + \Psi_g(A_0, A_1, B_0, B_1) \\ \leq \Psi_g(\overline{H}) + \Psi_g(G, \{H_S\}_S, A_0, A_1, B_0, B_1) \end{aligned}$$

Similarly, $\overline{H}$ is a list of $N - \sum_S n_S$ entries, and thus $\Psi_g(\overline{H}) \leq N - \sum_S n_S$. Thus:

$$\begin{aligned} (N - \sum_S n_S) + \sum_S \Psi_g(H_S) + \Psi_g(A_0, A_1, B_0, B_1) \\ \leq (N - \sum_S n_S) + \Psi_g(G, \{H_S\}_S, A_0, A_1, B_0, B_1) \end{aligned}$$

Subtracting $N - \sum_S n_S$ from both sides gives the desired bound.

This proves the lemma for our variant of GDist that has no special processing for 1-way queries. In Section 4.1 we modified GDist to eliminate H_S variables corresponding to 1-way queries. Instead, these values are secret shared, and the shares placed into two different 2-way H_S random variables. The preceding proof still holds, after making the following change: Define H to be not only the random oracle, but also the coin-tosses used for the secret sharing steps in (the modified) GDist. When a random share is included in H_S, that share is punctured (from the new region of H) and absent in $\overline{H}$; when the other share of the form $R \oplus H(S, X)$ is included in H_S, we also puncture $H(S, X)$ from H in the usual way.

With these modifications, it is still true that all of H (now representing the random oracle's truth table and all randomness from the sharing algorithm) can be computed from $(\overline{H}, G, \{H_S\}_S, A_0, A_1, B_0, B_1)$. The rest of the proof follows identical logic. □

5.4 No 3-way Queries

In this section we show that 3-way queries are always really 4-queries, as they can be computed with high probability in all 4 cases. The intuition is that if a query $H(S, X)$ could be computed in 3 out of 4 input cases, then X functions like the 0-output wire of a privacy-free ([FNO15]) AND gate. Furthermore, Ev makes only non-adaptive random oracle queries, so X is computed without using the random oracle at all. As we show, a privacy-free AND gate that avoids the random oracle is too good to be true.

Lemma 22. *Let (S, X^*) be any oracle query with $|S| = 3$. Then X^* can be computed as a deterministic function of (A_i, B_j, G) (in the* GDist(g)*), for* all *(i, j), except with probability at most $\lambda \ln(2) (2\epsilon_S + 4\tilde{\phi}(\epsilon_S))$, where ϵ_S is the scheme's negligible simulation error.*

Proof. When $(i,j) \in S$ this is trivial, because the honest (i,j)-evaluator makes this query. More precisely, $\Psi_g(X^* \mid G, A_i, B_j) = 0$ for $(i,j) \in S$. (The entropy is zero because we demand that the coordinate queries property holds perfectly.) Our goal is to show that the same expression $\Psi_g(X^* \mid G, A_i, B_j)$ is negligible for $(i,j) \notin S$. The intuition is that if the two entropies were significantly different, it would mean that G leaks information about some of the wire labels, which is not allowed.

Hereafter we take $(i,j) \notin S$. By input authenticity (Section 5.1),

$$\Psi_{1-i,j}(B_{1-j}; G \mid A_{1-i}, B_j) = 0.$$

Regardless of whether wire labels are uncorrelated or correlated by free-XOR, we have that B_{1-j} is independent of (A_{1-i}, B_j); that is, $\Psi_{1-i,j}(B_{1-j}; A_{1-i}, B_j) = 0$. Adding these together, along with the one-sided chain rule, gives

$$\Psi_{1-i,j}(B_{1-j}; A_{1-i}, B_j \mid G) = 0$$

Applying the one-sided chain rule again, we get:

$$0 \leq \Psi_{1-i,j}(B_{1-j}; A_{1-i}, B_j \mid G) = \Psi_{1-i,j}(B_{1-j}; A_{1-i}, B_j, G) - \Psi_{1-i,j}(B_{1-j}; G) \leq 0.$$

In other words, when the garbled gate G is public, learning wire labels A_{1-i}, B_j tells you nothing about the complementary label B_{1-j}. By a similar argument, we also have $\Psi_{i,1-j}(A_{1-i}; A_i, B_{1-j} \mid G) = 0$.

Using Lemma 19, we can translate these entropy bounds to the real $(\textsc{GDist}(g))$ garbling distribution. With simulation error ϵ_S and using that each wire label comes from a domain of size 2^λ, Lemma 19 gives

$$\Psi_g(B_{1-j}; A_{1-i}, B_j \mid G) \leq \epsilon_S + 2\tilde{\phi}(\epsilon_S) \quad \text{and} \quad \Psi_g(A_{1-i}; A_i, B_{1-j} \mid G) \leq \epsilon_S + 2\tilde{\phi}(\epsilon_S).$$

We now use these versions of input authenticity to upper bound $\Psi_g(X^* \mid G)$. First, because X^* can be computed from G, A_{1-i}, and B_j, we have

$$\Psi_g(B_{1-j}; A_{1-i}, B_j, X^* \mid G) = \Psi_g(B_{1-j}; A_{1-i}, B_j \mid G).$$

Then,

$$\begin{aligned}\Psi_g(B_{1-j}; X^* \mid G) &= \Psi_g(B_{1-j}; A_{1-i}, B_j, X^* \mid G) - \Psi_g(B_{1-j}; A_{1-i}, B_j \mid X^*, G) \\ &\leq \epsilon_S + 2\tilde{\phi}(\epsilon_S),\end{aligned}$$

using the one-sided chain rule. Therefore, B_{1-j} on its own says very little about X^*. Similarly, $\Psi_g(A_{1-i}; A_i, B_{1-j}, X^* \mid G) = \Psi_g(A_{1-i}; A_i, B_{1-j} \mid G)$, so

$$\begin{aligned}\Psi_g(A_{1-i}; X^* \mid B_{1-j}, G) &= \Psi_g(A_{1-i}; B_{1-j}, X^* \mid G) - \Psi_g(A_{1-i}; B_{1-j} \mid G) \\ &\leq \Psi_g(A_{1-i}; A_i, B_{1-j}, X^* \mid G) \leq \epsilon_S + 2\tilde{\phi}(\epsilon_S),\end{aligned}$$

where we have also used that adding A_i to the mutual information can only increase it. Therefore, A_{1-i} says very little about X^*, even given B_{1-j}.

If neither A_{1-i} nor B_{1-j} contain information about X^*, yet A_{1-i}, B_{1-j}, G together are enough to find X^*, then the information about X^* must be in the garbled gate G that we've been conditioning

on. More formally,

$$\begin{aligned}\Psi_g(X^* \mid G) &\le \Psi_g[A_{1-i}, B_{1-j}, X^*; X^* \mid G)\\ &= \Psi_g(A_{1-i}, B_{1-j}; X^* \mid G)\\ &= \Psi_g(B_{1-j}; X^* \mid G) + \Psi_g[A_{1-i}; X^* \mid B_{1-j}, G)\\ &\le 2\epsilon_S + 4\tilde{\phi}(\epsilon_S),\end{aligned}$$

using that X^* can be computed from G, A_{1-i}, and B_{1-j}, as well as the one-sided chain rule. Therefore, X^* can be computed from G, (even with only A_i and B_j for $(i,j) \notin S$), except with the correctness error given by Lemma 9. □

Corollary 23. *A garbling scheme with non-adaptive, coordinated queries can be modified to eliminate 3-way queries, at a cost of increasing its simulation error by at most* $\lambda \ln(2)(2\epsilon_S + 4\tilde{\phi}(\epsilon_S))$.

5.5 No "Cross Queries" Without Free-XOR

When wire labels have free-XOR correlation, we have $A_i \oplus B_j = A_{1-i} \oplus B_{1-j}$, and therefore an oracle query of the form $H(A_i \oplus B_j)$ is a 2-way query between evaluators (i,j) and $(1-i, 1-j)$. However, when wire labels are uncorrelated, we show that such a 2-way query is not possible. Or, more precisely, any such 2-way query is actually a 4-way query:

Lemma 24. *Suppose a gate-garbling scheme has uncorrelated wire labels. Let* (S, X^*) *be any oracle query with* S *of the form* $\{(i,j), (1-i, 1-j\}$. *Then* X^* *can be computed as a deterministic function of* (A_i, B_j, G) *(in the* GDIST(g)*), for* all (i,j)*, except with probability at most* $\lambda \ln(2)\,(2\epsilon_S + 4\tilde{\phi}(\epsilon_S))$, *where* ϵ_S *is the scheme's negligible simulation error.*

Proof. Without loss of generality, we take $S = \{(0,0), (1,1)\}$. When wire labels are uncorrelated, we have $\Psi_{i,j}(A_0, B_0; A_1, B_1) = 0$. (This mutual entropy is 1 with free-XOR wire labels.)

In ideal world (0,0), complementary wire labels A_1, B_1 are sampled independently of the evaluator's view (G, A_0, B_0), and thus:

$$\Psi_{0,0}(A_1, B_1; G \mid A_0, B_0) = 0.$$

Then by the one-sided chain rule we have:

$$\Psi_{0,0}(A_1, B_1; G, A_0, B_0) = \Psi_{0,0}(G; A_1, B_1 \mid A_0, B_0) + \Psi_{0,0}(A_0, B_0; A_1, B_1) = 0.$$

X^* can be computed deterministically as a function of G, A_0, B_0, and so

$$\Psi_{0,0}(X^* \mid G, A_0, B_0) = 0.$$

In other words, G, A_0, B_0 already exhaust all the entropy of X^*. Including A_1, B_1 in the mutual entropy has no effect, leading to:

$$\Psi_{0,0}(X^*; A_1, B_1 \mid G, A_0, B_0) = 0.$$

Again applying the one-sided chain rule, we get:

$$\Psi_{0,0}(X^*, G, A_0, B_0; A_1, B_1) = \Psi_{0,0}(G, A_0, B_0; A_1, B_1) + \Psi_{0,0}(X^*; A_1, B_1 \mid G, A_0, B_0) = 0.$$

Removing A_0, B_0 from the expression can only decrease entropy, so we have

$$\Psi_{0,0}(X^*, G; A_1, B_1) = 0.$$

Again applying the one-sided chain rule, we get:

$$0 = \Psi_{0,0}(X^*, G; A_1, B_1) = \Psi_{0,0}(G; A_1, B_1) + \Psi_{0,0}(X^*; A_1, B_1 \mid G)$$

so we conclude that $\Psi_{0,0}(G; A_1, B_1) = 0$.

We can now begin to bound the entropy of X^*, as:

$$\Psi_g(X^* \mid G) = \Psi_g(X^* \mid G, A_1, B_1) + \Psi_g(X^*; A_1, B_1 \mid G)$$

Using Lemma 19, these bounds for Ψ_g hold also for $\Psi_{i,j}$, up to an additive term $\epsilon_S + 2\tilde{\phi}(\epsilon_S)$; thus:

$$\Psi_g(X^* \mid G) \leq \Psi_{1,1}(X^* \mid G, A_1, B_1) + \Psi_{0,0}(X^*; A_1, B_1 \mid G) + 2(\epsilon_S + 2\tilde{\phi}(\epsilon_S))$$

But the $\Psi_{1,1}$ term is zero because X^* is computed deterministically by the evaluator as a function of G, A_1, B_1 in SimDist$(1, 1)$. And we previously showed that the $\Psi_{0,0}$ term is also zero. Thus, $\Psi_g(X^* \mid G) \leq 2(\epsilon_S + 2\tilde{\phi}(\epsilon_S))$. In other words, X^* is almost entirely determined by G alone. Therefore, X^* can be computed from G alone, (for any choice of A_i and B_j), except with the correctness error given by Lemma 9. □

Corollary 25. *A garbling scheme with non-adaptive, coordinated queries and uncorrelated wire labels can be modified to have no 2-way queries of the form $S = \{(i, j), (1 - i, 1 - j)\}$, at a cost of increasing its simulation error by at most $\lambda \ln(2)(2\epsilon_S + 4\tilde{\phi}(\epsilon_S))$.*

6 Proving Lower Bounds with a Shannon-Inequality Solver

We obtain our final lower bound by enlisting the help of an automated prover for Shannon-type inequalities, called `Citip` [Glä15]. The input to Citip is a list of information inequalities, each one an affine function of terms of the form `H(X)`, `H(X|Y)`, `I(X;Y)`, etc. There is also syntactic sugar for common properties:

- "`X <- Y`" means that X is fully determined by Y, i.e., $\Psi(X \mid Y) = 0$.
- "`X . Y . Z,W`" means that X, Y, and the pair (Z, W) are all independent. That is, $\Psi(X, Y, Z, W) = \Psi(X) + \Psi(Y) + \Psi(Z, W)$.

One may then provide a final Shannon bound and ask Citip to prove (or find a counterexample) that the bound follows from the given ones.

Internally, Citip creates a big linear program based on all of our constraints and the basic (axiomatic) Shannon-type inequalities. This linear program has one variable for every non-empty *subset* of random variables, representing their entropy. E.g. if the random variables are X and Y then the linear program will contain 3 variables, representing $\Psi(X)$, $\Psi(Y)$, and $\Psi(X, Y)$. There are a similarly exponential number of basic Shannon-type inequalities, so this process is only practical for relatively small numbers of random variables. Citip then requests a linear programming solver to find a violation of the requested bound, if one exists; we used the Clp solver [FVR+23].

Syntax of Citip input: We have modified Citip to include the following new features useful for proving cryptographic bounds.[3]

[3] Our modifications can be found at `https://github.com/ldr709/Citip`.

- We add support for inequalities over different distributions. Our lower bounds use 5 distributions: the four SimDist(i,j) distributions, plus a GDist(g) distribution for an arbitrarily chosen $g \in \mathcal{G}$ (a single g suffices for our result). Information bounds in a distribution can be declared using (for example) `H01(...)` or `IR(..;..)`, referring to $\Psi_{i,j}$ and Ψ_g.

- We add support for formal variables in the bounds. E.g. "`H(X|Y) <= eps`" allows for some violation of functional dependence, bounded by a formal variable `eps`.

- We provide a way to translate information bounds between distributions, following Lemma 19. Writing

  ```
  indist 00 :  X,Y,Z ; 01 :  U,V,W approx eps,eps_X,eps_Y,eps_Z
  ```

  means that any information bound on (only) variables X, Y, Z that holds in SimDist$(0,0)$ also holds with respect to U, V, W in SimDist$(0,1)$, but with the introduction of an error term. Recall that the error term in Lemma 19 has the form $\epsilon \log(N) + 2\tilde{\phi}(\epsilon)$, where N is the cardinality of the support of the bound in question. Thus, the error terms depend on which subset of (X, Y, Z) is being considered. In our syntax, `eps` refers to the fixed term $\tilde{\phi}(\epsilon)$, while `eps_X` refers to $\epsilon \log(N)$, where N is the cardinality of X, and so on. Thus, these error terms become formal variables in the underlying linear program.

- We added a way to ask Citip to prove a bound parametric in some unknown coefficients (`c1`, `c2`, ...). Requesting a bound like "`HR(G) >= c1 + c2 eps`" causes it to optimize for the maximum '`c1`, `c2`' (in that order) such that this holds. If `eps` then represents simulation error, `c2` will then be some negative value representing how much increased simulation error decreases the bound. We eventually ask Citip to prove (for example) a lower bound of the form, and include coefficients for each `eps_X` formal variable.

- We use the dual solution returned by the linear solver to output a proof of the bound, consisting of a linear combination of known inequalities that gives the desired bound.

- We included an optimization for functional dependence relationships, to reduce the instance size for the solver. For example, "`01 :  X <- Y implicit`" means that in SimDist$(0,1)$, X is completely determined by Y, and the "`implicit`" says to use this fact to not consider any entropy that contains Y but not X. That is, for any (set of) random variables Z we have $\Psi_{01}(Y, Z) = \Psi_{01}(X, Y, Z)$, so we eliminate the former by replacing it with the latter in all bounds.

Variables for our bounds: We provide Citip with a list of information bounds involving the following variables:

- Wire labels `A0`, `A1`, `B0`, `B1`, `C0`, `C1`. Following Remark 18 we assume that $k = 0$ always, so `C0` in an ideal world may correspond with `C1` in the real world. In the case of free-XOR, we use `D` to denote the global offset Δ.

- Garbled gate `G`.

- Random oracle responses H_S, as defined in Section 4. Following the optimization in Section 4.1 and Corollary 23, we only need variables representing 2-way and 4-way queries. Our Citip variable names reflect the "archetypal" 2-way queries in free-XOR, so (for example) `HA_0` refers to $H(A_0)$, representing all 2-way query responses: H_S for $S = \{(0,0),(0,1)\}$.

Similarly, `HAB_1` refers to $H(A_0 \oplus B_1) = H(A_1 \oplus B_0)$, representing $S = \{(0,1),(1,0)\}$. The variable representing 4-way queries is called `H_4`.

- As an optimization we also define a variable `inputs_and_H_4`, which represents $(A_0, A_1, B_0, B_1, H_4)$. These individual random variables are never used directly in the real world $\text{GDIST}(g)$ distribution bounds, so by using `inputs_and_H_4` we allow Citip to track fewer variables.

6.1 The Free-XOR Lower Bound

Our main theorem is the following:

Theorem 26. *If a gate garbling scheme has non-adaptive, coordinated queries, is strongly secure with respect to the class of AND-gates ($\mathcal{G}_{\mathsf{and}} = \{(i,j) \to (i \oplus \pi_A) \wedge (j \oplus \pi_B) \oplus \pi_C \mid \pi_A, \pi_B, \pi_C \in \{0,1\}\}$), then its garbled gates have size at least $1.5\lambda - \mathsf{negl}(\lambda)$.*

Corollary 27. *The three-halves garbling scheme of Rosulek & Roy [RR21], which supports free-XOR and has garbled gates consisting of $1.5\lambda + O(1)$ bits, is size-optimal for this setting, up to a small additive constant.*

of Theorem 26. The bound is proven using our modified Citip software, as described above. The complete Citip input file is shown in Section A.1 and also available at `https://github.com/rosulek/garbling-lowerbounds`. Here we simply describe the meaning of this input file.

> *In the supplementary material for this submission we provide all Citip input/output files, as well as a simpler script that can be used to verify the output.*

The information bounds in the Citip input file can be grouped as follows:

- Lines 1-2: `inputs_and_H_4` is equivalent to $(A_0, A_1, B_0, B_1, \Delta, H_4)$.
- Line 3: Lemma 21
- Line 4-31: In every ideal distribution, every wire label has entropy 1 (block).
- Lines 32-40: Any two of (A_0, A_1, Δ) completely determine the third. This is true for all wire-label pairs (A's, B's, and C's), and in all ideal distributions.
- Lines 41-44 reflect the sampling of (A_1, B_1, C_1) – and implicitly, Δ — via $\mathsf{CompLbl}_{\mathsf{free\text{-}xor}}$ in $\text{SIMDIST}(0,0)$. Each of the complementary labels is individually independent of the visible wire labels (A_0, B_0, C_0).
- Line 45: In $\text{SIMDIST}(0,0)$, the garbled output is completely determined by the evaluator's view (input labels, garbled gate, and relevant random oracle responses).
- Lines 46-47 are the input/output authenticity properties described in Section 5.1.
- Line 48: $\text{SIMDIST}(0,0)$ is indistinguishable from $\text{GDIST}(g)$; Lemma 19.
- Lines 49-72 are copies of lines 41-48, adapted to the three other ideal distributions.
- Lines 72-84: The class of AND-gates $\mathcal{G}_{\mathsf{and}}$ has the following property: for all $(i,j) \neq (i',j')$, there is a choice of g such that $g(i,j) = g(i',j')$ and a choice of g such that $g(i,j) \neq g(i',j')$. We can apply Lemma 20 to argue that entropy bounds on $\Psi_{i,j}$ can be translated to $\Psi_{i',j'}$, with the roles of C_0, C_1 preserved (corresponding to $g(i,j) = g(i',j')$) or swapped (corresponding to $g(i,j) \neq g(i',j')$).

Given these bounds, we ask Citip to prove a lower bound on the entropy of G. Specifically, we ask it to prove a bound of the form

$$\Psi_g(G \mid \texttt{inputs\_and\_H\_4}) \geq c_1 + \textstyle\sum_i c_i \cdot \epsilon_i,$$

where the ϵ_i's are the various slack values from Lemma 19.

Citip concludes that

$$\Psi_g(G \mid \texttt{inputs\_and\_H\_4}) \geq 1.5 - 8.5\epsilon_W - 10\epsilon_0.$$

If ϵ is the simulation error of the garbling scheme, then our variable $\epsilon_W = 4\epsilon$ (the error term involving distributions over input wire labels, which are determined by 4 values: A_0, B_0, C_0, Δ; for non-free-XOR garbling, $\epsilon_W = 6\epsilon$), and $\epsilon_0 = \tilde{\phi}(\epsilon)$. Both of these are negligible if the garbling scheme is secure. □

Interpreting and verifying the output. The output of Citip is long list of information inequalities, each of which follows directly from the bounds in the input file and the axioms of Shannon-type bounds. Adding up these inequalities results in the bound on the last line. The earlier lines of the Citip output tend to be generic Shannon inequalities, and the later lines are constraints from the input. The prover appears to be using all classes of constraints, though many individual constraints go unused due to symmetry. Of course, it could be using constraints unnecessarily, but in our experience it fails to find a lower bound if the input constraints are weakened significantly. Thus, we believe that the conditions in Theorem 26 are more-or-less minimal.

At https://github.com/rosulek/garbling-lowerbounds, we provide a simple and self-contained script that can verify such output.

6.2 The Non-Free-XOR Lower Bound

Next, we prove the following:

Theorem 28. *If a gate garbling scheme has non-adaptive, coordinated queries, is strongly secure with respect to the class of AND-gates ($\mathcal{G}_{\mathsf{and}} = \{(i,j) \to (i \oplus \pi_A) \wedge (j \oplus \pi_B) \oplus \pi_C \mid \pi_A, \pi_B, \pi_C \in \{0,1\}\}$), and uses **uncorrelated wire labels**, then its garbled gates have size at least $2\lambda - \mathsf{negl}(\lambda)$.*

Proof. The Citip input file for this case is shown in Section A.2 and also available at https://github.com/rosulek/garbling-lowerbounds. It is mostly identical to the previous lower bound, with only the following change, reflecting the lack of free-XOR:

- Following Corollary 25, we do not include random variables H_S for the "cross queries" of the form $S = \{(0,0),(1,1)\}$ and $S = \{(0,1),(1,0)\}$.
- We simplify the description of wire label distributions, reflecting the use of $\mathsf{CompLbl}_{\mathsf{plain}}$ in $\textsc{SimDist}(i,j)$. Lines 28, 33, 38, 43 simply say that (A_0, B_0, C_0) are independent of (A_1, B_1, C_1).

Citip concludes with the following bound:

$$\Psi_g(G \mid \texttt{inputs\_and\_H\_4}) \geq 2 - 9.6\overline{6}\epsilon_W - 11.6\overline{6}\epsilon_0.$$

□

Theorem 29. *If a gate garbling scheme has non-adaptive, coordinated queries, is strongly secure with respect to XOR-gates ($\mathcal{G} = \{\oplus\}$) and uses* ***uncorrelated wire labels****, then its garbled gates have size at least $\lambda - \mathsf{negl}(\lambda)$.*

Proof. The Citip input file for this case is shown in Section A.3 and also available at `https://github.com/rosulek/garbling-lowerbounds`. It is mostly identical to the previous lower bound, with only the following change, reflecting the choice of gate:

- The class of AND gates has the property that for any pairs of inputs $(i, j), (i', j')$, there is a gate giving the same output, and a gate giving opposite output. XOR gates do not have this property. Instead, it is public knowledge that $g(i, j) = g(i', j') \oplus i \oplus i' \oplus j \oplus j'$. Thus, any information bound on $\Psi_{i,j}$ also holds (up to an error term) with respect to $\Psi_{i',j'}$, but with output labels (C_0, C_1) swapped iff $i \oplus i' \oplus j \oplus j' = 1$. Lines 48-53 reflect this condition.

Citip concludes with the following bound:

$$\Psi_g(G \mid \texttt{inputs\_and\_H\_4}) \geq 1 - 4.7\overline{7}\epsilon_W - 5.4\overline{4}\epsilon_0.$$

(The Citip output also includes a term $4\text{E-}18 \cdot \epsilon_H$, which we assume to be a rounding error.) Interestingly, this is by far the longest Citip proof of our three results. □

Corollary 30. *The garbling scheme of Gueron, Lindell, Nof, and Pinkas [GLNP15], which has garbled AND-gates consisting of $1.5\lambda + O(1)$ bits and garbled XOR-gates consisting of $\lambda + O(1)$ bits, is size-optimal up to a small additive constant.*

References

[BHR12] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012*, pages 784–796. ACM Press, October 2012.

[BK24] Chunghun Baek and Taechan Kim. Can we beat three halves lower bound?: (Im)possibility of reducing communication cost for garbled circuits. Cryptology ePrint Archive, Report 2024/803, 2024.

[BMR90] Donald Beaver, Silvio Micali, and Phillip Rogaway. The round complexity of secure protocols (extended abstract). In *22nd ACM STOC*, pages 503–513. ACM Press, May 1990.

[BMR16] Marshall Ball, Tal Malkin, and Mike Rosulek. Garbling gadgets for Boolean and arithmetic circuits. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 565–577. ACM Press, October 2016.

[BO09] Amos Beimel and Ilan Orlov. Secret sharing and non-Shannon information inequalities. In Omer Reingold, editor, *TCC 2009*, volume 5444 of *LNCS*, pages 539–557. Springer, Berlin, Heidelberg, March 2009.

[Cov99] Thomas M Cover. *Elements of information theory*. John Wiley & Sons, 1999.

[Csi95] László Csirmaz. The size of a share must be large. In Alfredo De Santis, editor, *EUROCRYPT'94*, volume 950 of *LNCS*, pages 13–22. Springer, Berlin, Heidelberg, May 1995.

[FLZ24] Lei Fan, Zhenghao Lu, and Hong-Sheng Zhou. Column-wise garbling, and how to go beyond the linear model. Cryptology ePrint Archive, Report 2024/415, 2024.

[FNO15] Tore Kasper Frederiksen, Jesper Buus Nielsen, and Claudio Orlandi. Privacy-free garbled circuits with applications to efficient zero-knowledge. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 191–219. Springer, Berlin, Heidelberg, April 2015.

[FVR+23] John Forrest, Stefan Vigerske, Ted Ralphs, John Forrest, Lou Hafer, jpfasano, Haroldo Gambini Santos, Jan-Willem, Matthew Saltzman, Bjarni Kristjansson, h-i gassmann, Alan King, Arevall, Pierre Bonami, Ruan Luies, Samuel Brito, and to st. COIN-OR/Clp: Release releases/1.17.9, October 2023.

[Glä15] Thomas Gläßle. Citip: information theoretic inequality prover, 2015. Obtained from https://github.com/coldfix/Citip.

[GLNP15] Shay Gueron, Yehuda Lindell, Ariel Nof, and Benny Pinkas. Fast garbling of circuits under standard assumptions. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *ACM CCS 2015*, pages 567–578. ACM Press, October 2015.

[Imp95] Russell Impagliazzo. A personal view of average-case complexity. In *Proceedings of the Tenth Annual Structure in Complexity Theory Conference, Minneapolis, Minnesota, USA, June 19-22, 1995*, pages 134–147, 1995.

[KKS16] Carmen Kempka, Ryo Kikuchi, and Koutarou Suzuki. How to circumvent the two-ciphertext lower bound for linear garbling schemes. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 967–997. Springer, Berlin, Heidelberg, December 2016.

[Kol05] Vladimir Kolesnikov. Gate evaluation secret sharing and secure one-round two-party computation. In Bimal K. Roy, editor, *ASIACRYPT 2005*, volume 3788 of *LNCS*, pages 136–155. Springer, Berlin, Heidelberg, December 2005.

[KP17] Yashvanth Kondi and Arpita Patra. Privacy-free garbled circuits for formulas: Size zero and information-theoretic. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 188–222. Springer, Cham, August 2017.

[KS08] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfsdóttir, and Igor Walukiewicz, editors, *ICALP 2008, Part II*, volume 5126 of *LNCS*, pages 486–498. Springer, Berlin, Heidelberg, July 2008.

[LGW24] Ruiyang Li, Chun Guo, and Xiao Wang. Towards optimal garbled circuits in the standard model. Cryptology ePrint Archive, Paper 2024/1907, 2024.

[NPS99] Moni Naor, Benny Pinkas, and Reuban Sumner. Privacy preserving auctions and mechanism design. In *Proceedings of the 1st ACM Conference on Electronic Commerce*, pages 129–139, New York, NY, USA, 1999. ACM.

[PPD07] Rethnakaran Pulikkoonattu, Etienne Perron, and Suhas Diggavi. Xitip: information theoretic inequalities prover, 2007. https://xitip.epfl.ch/.

[PSSW09] Benny Pinkas, Thomas Schneider, Nigel P. Smart, and Stephen C. Williams. Secure two-party computation is practical. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 250–267. Springer, Berlin, Heidelberg, December 2009.

[PV10] Carles Padró and Leonor Vázquez. Finding lower bounds on the complexity of secret sharing schemes by linear programming. In Alejandro López-Ortiz, editor, *LATIN 2010*, volume 6034 of *LNCS*, pages 344–355. Springer, Berlin, Heidelberg, April 2010.

[RR21] Mike Rosulek and Lawrence Roy. Three halves make a whole? Beating the half-gates lower bound for garbled circuits. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 94–124, Virtual Event, August 2021. Springer, Cham.

[Shi17] M. E. Shirokov. Tight uniform continuity bounds for the quantum conditional mutual information, for the holevo quantity, and for capacities of quantum channels. *Journal of Mathematical Physics*, 58(10):102202, 2017.

[Win16] Andreas Winter. Tight uniform continuity bounds for quantum entropies: Conditional entropy, relative entropy distance and energy constraints. *Communications in Mathematical Physics*, 347(1):291–313, Oct 2016.

[WM17] Yongge Wang and Qutaibah M. Malluhi. Reducing garbled circuit size while preserving circuit gate privacy. Cryptology ePrint Archive, Report 2017/041, 2017.

[XHX24] Fei Xu, Honggang Hu, and Changhong Xu. Bitwise garbling schemes — a model with $\frac{3}{2}\kappa$-bit lower bound of ciphertexts. Cryptology ePrint Archive, Paper 2024/1532, 2024.

[Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167. IEEE Computer Society Press, October 1986.

[ZRE15] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole - reducing data transfer in garbled circuits using half gates. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 220–250. Springer, Berlin, Heidelberg, April 2015.

A Citip Input/Outputs

This section contains Citip input and output files the prove our lower bounds. The files are also available at https://github.com/rosulek/garbling-lowerbounds.

A.1 AND-gate with Free-XOR

Citip input:

```
00,01,10,11: A_0,A_1,B_0,B_1,D,H_4 <- inputs_and_H_4 implicit
00,01,10,11: inputs_and_H_4 <- A_0,A_1,B_0,B_1,D,H_4 implicit
HR(HA_0,HA_1,HB_0,HB_1,HAB_0,HAB_1 | inputs_and_H_4) >= HR(HA_0) + HR(HA_1) + HR(HB_0) + HR(
    HB_1) + HR(HAB_0) + HR(HAB_1)
H00(A_0) = 1
H01(A_0) = 1
H10(A_0) = 1
H11(A_0) = 1
H00(A_1) = 1
H01(A_1) = 1
H10(A_1) = 1
H11(A_1) = 1
H00(B_0) = 1
H01(B_0) = 1
H10(B_0) = 1
H11(B_0) = 1
H00(B_1) = 1
H01(B_1) = 1
H10(B_1) = 1
H11(B_1) = 1
H00(C_0) = 1
H01(C_0) = 1
H10(C_0) = 1
H11(C_0) = 1
H00(C_1) = 1
H01(C_1) = 1
H10(C_1) = 1
H11(C_1) = 1
H00(D) = 1
H01(D) = 1
H10(D) = 1
H11(D) = 1
00,01,10,11: A_0,A_1,D <- A_0,A_1 implicit
00,01,10,11: A_0,A_1,D <- A_0,D implicit
00,01,10,11: A_0,A_1,D <- A_1,D implicit
00,01,10,11: B_0,B_1,D <- B_0,B_1 implicit
00,01,10,11: B_0,B_1,D <- B_0,D implicit
00,01,10,11: B_0,B_1,D <- B_1,D implicit
00,01,10,11: C_0,C_1,D <- C_0,C_1 implicit
00,01,10,11: C_0,C_1,D <- C_0,D implicit
00,01,10,11: C_0,C_1,D <- C_1,D implicit
00: D . A_0,B_0,C_0
00: A_1 . A_0,B_0,C_0
00: B_1 . A_0,B_0,C_0
00: C_1 . A_0,B_0,C_0
00: C_0 <- G,A_0,B_0,H_4,HA_0,HB_0,HAB_0
I00(A_0,A_1,B_0,B_1,C_0,C_1; G,H_4,HA_0,HB_0,HAB_0 | A_0,B_0,C_0) <= 0
I00(A_0,A_1,B_0,B_1,D; G,H_4,HA_0,HB_0,HAB_0 | A_0,B_0) <= 0
indist R: C_0,C_1,G,inputs_and_H_4,HA_0,HB_0,HAB_0 ; 00: C_0,C_1,G,inputs_and_H_4,HA_0,HB_0,
    HAB_0 approx eps, eps_W,, eps_G, eps_W + eps_H,,,
01: D . A_0,B_1,C_0
01: A_1 . A_0,B_1,C_0
01: B_0 . A_0,B_1,C_0
01: C_1 . A_0,B_1,C_0
01: C_0 <- G,A_0,B_1,H_4,HA_0,HB_1,HAB_1
I01(A_0,A_1,B_0,B_1,C_0,C_1; G,H_4,HA_0,HB_1,HAB_1 | A_0,B_1,C_0) <= 0
I01(A_0,A_1,B_0,B_1,D; G,H_4,HA_0,HB_1,HAB_1 | A_0,B_1) <= 0
```

```
indist R: C_0,C_1,G,inputs_and_H_4,HA_0,HB_1,HAB_1 ; 01: C_1,C_0,G,inputs_and_H_4,HA_0,HB_1,
    HAB_1 approx eps, eps_W,, eps_G, eps_W + eps_H,,,
10: D . A_1,B_0,C_0
10: A_0 . A_1,B_0,C_0
10: B_1 . A_1,B_0,C_0
10: C_1 . A_1,B_0,C_0
10: C_0 <- G,A_1,B_0,H_4,HA_1,HB_0,HAB_1
I10(A_0,A_1,B_0,B_1,C_0,C_1; G,H_4,HA_1,HB_0,HAB_1 | A_1,B_0,C_0) <= 0
I10(A_0,A_1,B_0,B_1,D; G,H_4,HA_1,HB_0,HAB_1 | A_1,B_0) <= 0
indist R: C_0,C_1,G,inputs_and_H_4,HA_1,HB_0,HAB_1 ; 10: C_1,C_0,G,inputs_and_H_4,HA_1,HB_0,
    HAB_1 approx eps, eps_W,, eps_G, eps_W + eps_H,,,
11: D . A_1,B_1,C_0
11: A_0 . A_1,B_1,C_0
11: B_0 . A_1,B_1,C_0
11: C_1 . A_1,B_1,C_0
11: C_0 <- G,A_1,B_1,H_4,HA_1,HB_1,HAB_0
I11(A_0,A_1,B_0,B_1,C_0,C_1; G,H_4,HA_1,HB_1,HAB_0 | A_1,B_1,C_0) <= 0
I11(A_0,A_1,B_0,B_1,D; G,H_4,HA_1,HB_1,HAB_0 | A_1,B_1) <= 0
indist R: C_0,C_1,G,inputs_and_H_4,HA_1,HB_1,HAB_0 ; 11: C_1,C_0,G,inputs_and_H_4,HA_1,HB_1,
    HAB_0 approx eps, eps_W,, eps_G, eps_W + eps_H,,,
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HA_0,H_4 ; 01: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HA_0,H_4
    approx 2 eps, 2 eps_W,,,,,,, 2 eps_G, 2 eps_H,
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HA_0,H_4 ; 01: C_1,C_0,A_0,A_1,B_0,B_1,D,G,HA_0,H_4
    approx 2 eps, 2 eps_W,,,,,,, 2 eps_G, 2 eps_H,
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HB_0,H_4 ; 10: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HB_0,H_4
    approx 2 eps, 2 eps_W,,,,,,, 2 eps_G, 2 eps_H,
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HB_0,H_4 ; 10: C_1,C_0,A_0,A_1,B_0,B_1,D,G,HB_0,H_4
    approx 2 eps, 2 eps_W,,,,,,, 2 eps_G, 2 eps_H,
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HAB_0,H_4 ; 11: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HAB_0,H_4
     approx 2 eps, 2 eps_W,,,,,,, 2 eps_G, 2 eps_H,
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HAB_0,H_4 ; 11: C_1,C_0,A_0,A_1,B_0,B_1,D,G,HAB_0,H_4
     approx 2 eps, 2 eps_W,,,,,,, 2 eps_G, 2 eps_H,
indist 01: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HAB_1,H_4 ; 10: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HAB_1,H_4
     approx 2 eps, 2 eps_W,,,,,,, 2 eps_G, 2 eps_H,
indist 01: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HAB_1,H_4 ; 10: C_1,C_0,A_0,A_1,B_0,B_1,D,G,HAB_1,H_4
     approx 2 eps, 2 eps_W,,,,,,, 2 eps_G, 2 eps_H,
indist 01: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HB_1,H_4 ; 11: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HB_1,H_4
    approx 2 eps, 2 eps_W,,,,,,, 2 eps_G, 2 eps_H,
indist 01: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HB_1,H_4 ; 11: C_1,C_0,A_0,A_1,B_0,B_1,D,G,HB_1,H_4
    approx 2 eps, 2 eps_W,,,,,,, 2 eps_G, 2 eps_H,
indist 10: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HA_1,H_4 ; 11: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HA_1,H_4
    approx 2 eps, 2 eps_W,,,,,,, 2 eps_G, 2 eps_H,
indist 10: C_0,C_1,A_0,A_1,B_0,B_1,D,G,HA_1,H_4 ; 11: C_1,C_0,A_0,A_1,B_0,B_1,D,G,HA_1,H_4
    approx 2 eps, 2 eps_W,,,,,,, 2 eps_G, 2 eps_H,
prove HR(G | inputs_and_H_4) >= c1 + c2 eps_H + c3 eps_G + c4 eps_W + c5 eps
```

Citip output:

```
HR(G | inputs_and_H_4,HA_0,HA_1,HB_0,HB_1,HAB_0,HAB_1,C_0,C_1) >= 0
HR(C_1 | G,inputs_and_H_4,HA_0,HA_1,HB_0,HB_1,HAB_0,HAB_1,C_0) >= 0
IR(G; HA_0) >= 0
IR(G; HA_1 | inputs_and_H_4,HB_1) >= 0
0.5 IR(G; HB_0 | inputs_and_H_4,HA_1) >= 0
0.5 IR(G; HB_0 | inputs_and_H_4,HA_0,HAB_0) >= 0
IR(G; HB_1 | inputs_and_H_4) >= 0
IR(G; HAB_0 | inputs_and_H_4,HB_1) >= 0
0.5 IR(G; HAB_1 | inputs_and_H_4) >= 0
0.5 IR(G; HAB_1 | inputs_and_H_4,HB_1) >= 0
IR(G; C_0 | inputs_and_H_4,HA_0,HA_1,HB_0,HB_1,HAB_0,HAB_1,C_1) >= 0
IR(G; C_1 | inputs_and_H_4,HA_0,HA_1,HB_0,HB_1,HAB_0,HAB_1,C_0) >= 0
0.5 IR(inputs_and_H_4; HA_0 | G) >= 0
0.5 IR(inputs_and_H_4; HA_0 | G,HB_1,HAB_1) >= 0
IR(inputs_and_H_4; HA_1 | HB_1) >= 0
0.5 IR(inputs_and_H_4; HB_0 | HA_1) >= 0
0.5 IR(inputs_and_H_4; HB_0 | HA_0,HAB_0) >= 0
IR(inputs_and_H_4; HB_1 | HAB_0) >= 0
IR(inputs_and_H_4; HAB_0) >= 0
IR(inputs_and_H_4; HAB_1) >= 0
0.5 IR(HA_0; HA_1 | G,inputs_and_H_4,HB_1,HAB_0,C_1) >= 0
```

```
0.5 IR(HA_0; HA_1 | G,inputs_and_H_4,HB_1,HAB_1,C_1) >= 0
0.5 IR(HA_0; HB_0) >= 0
0.5 IR(HA_0; HB_0 | G,inputs_and_H_4,HA_1,HB_1,HAB_1,C_1) >= 0
0.5 IR(HA_0; HB_1 | G,HAB_1) >= 0
0.5 IR(HA_0; HB_1 | G,inputs_and_H_4,HB_0,HAB_0,C_1) >= 0
0.5 IR(HA_0; HAB_0 | G,inputs_and_H_4) >= 0
0.5 IR(HA_0; HAB_0 | G,inputs_and_H_4,HA_1,HB_0,HB_1,C_1) >= 0
0.5 IR(HA_0; HAB_1 | G) >= 0
0.5 IR(HA_0; HAB_1 | G,inputs_and_H_4,HA_1,HB_0,HB_1,HAB_0,C_1) >= 0
0.5 IR(HA_1; HB_0) >= 0
0.5 IR(HA_1; HB_0 | G,inputs_and_H_4,HA_0,HB_1,HAB_0,C_1) >= 0
IR(HA_1; HB_1) >= 0
IR(HA_1; HAB_0 | G,inputs_and_H_4,HB_1) >= 0
0.5 IR(HA_1; HAB_1 | G,inputs_and_H_4,HB_0) >= 0
0.5 IR(HA_1; HAB_1 | G,inputs_and_H_4,HB_1,HAB_0,C_1) >= 0
0.5 IR(HB_0; HB_1 | G,inputs_and_H_4,HAB_0,C_1) >= 0
0.5 IR(HB_0; HB_1 | G,inputs_and_H_4,HA_1,HAB_0,HAB_1,C_1) >= 0
0.5 IR(HB_0; HAB_0 | HA_0) >= 0
0.5 IR(HB_0; HAB_0 | G,inputs_and_H_4,HA_1,HAB_1,C_1) >= 0
0.5 IR(HB_0; HAB_1 | G,inputs_and_H_4) >= 0
0.5 IR(HB_0; HAB_1 | G,inputs_and_H_4,HA_1,C_1) >= 0
IR(HB_1; HAB_0) >= 0
0.5 IR(HB_1; HAB_1 | inputs_and_H_4) >= 0
0.5 IR(HB_1; HAB_1 | G,inputs_and_H_4,HA_1,HB_0,C_1) >= 0
0.5 IR(HAB_0; HAB_1 | G,inputs_and_H_4,HB_1,C_1) >= 0
0.5 IR(HAB_0; HAB_1 | G,inputs_and_H_4,HA_0,HA_1,HB_0,HB_1,C_1) >= 0
IR(C_0; C_1 | inputs_and_H_4,HA_0,HA_1,HB_0,HB_1,HAB_0,HAB_1) >= 0
0.5 I00(A_1,B_1,D,inputs_and_H_4; C_0 | A_0,B_0,H_4,G,HA_0,HB_0,HAB_0) >= 0
0.5 I01(A_1,B_0,D,inputs_and_H_4; C_0 | A_0,B_1,H_4,G,HA_0,HB_1,HAB_1) >= 0
0.5 I10(A_0,B_1,D; HA_1 | A_1,B_0) >= 0
0.5 I10(A_0,B_1,D; G | A_1,B_0,HA_1) >= 0
I10(A_0,B_1,D,inputs_and_H_4,C_1; HAB_1 | A_1,B_0,H_4,C_0,G,HA_1,HB_0) >= 0
0.5 I10(A_0,B_1,D; H_4 | A_1,B_0,G,HA_1) >= 0
0.5 I10(C_0; C_1 | A_1,B_0,H_4,G,HA_1,HB_0) >= 0
0.5 I10(C_1; HB_0 | A_1,B_0,H_4,G,HA_1) >= 0
0.5 I10(A_0,B_1,D,inputs_and_H_4,C_1; HB_0 | A_1,B_0,H_4,C_0,G,HA_1) >= 0
0.5 I11(A_0,B_0,D,inputs_and_H_4; G | A_1,B_1,H_4) >= 0
0.5 I11(A_0,B_0,D; G | A_1,B_1,HB_1) >= 0
0.5 I11(A_0,B_0,D; HB_1 | A_1,B_1) >= 0
0.5 I11(A_0,B_0,D; H_4 | A_1,B_1) >= 0
0.5 I11(A_0,B_0,D; H_4 | A_1,B_1,G,HB_1) >= 0
0.5 I11(A_0,B_0,D,inputs_and_H_4,C_1; HA_1 | A_1,B_1,H_4,C_0,G,HAB_0) >= 0
0.5 I11(A_0,B_0,D,inputs_and_H_4; HAB_0 | A_1,B_1,H_4,G) >= 0
0.5 I11(C_0; C_1 | A_1,B_1,H_4,G,HB_1) >= 0
0.5 I11(C_0; C_1 | A_1,B_1,H_4,G,HA_1,HAB_0) >= 0
I11(A_0,B_0,D,inputs_and_H_4,C_1; HA_1 | A_1,B_1,H_4,C_0,G,HB_1) >= 0
0.5 I11(C_1; HA_1 | A_1,B_1,H_4,G,HAB_0) >= 0
I11(A_0,B_0,D,inputs_and_H_4,C_1; HB_1 | A_1,B_1,H_4,C_0,G,HA_1,HAB_0) >= 0
I11(A_0,B_0,D,inputs_and_H_4,C_1; HAB_0 | A_1,B_1,H_4,C_0,G,HA_1,HB_1) >= 0
-HR(inputs_and_H_4) - HR(HA_0) - HR(HA_1) - HR(HB_0) - HR(HB_1) - HR(HAB_0) - HR(HAB_1) + HR
    (inputs_and_H_4,HA_0,HA_1,HB_0,HB_1,HAB_0,HAB_1) >= 0
H11(B_0) == 1
0.5 H10(B_1) == 0.5
0.5 H00(A_0,B_0,H_4,G,HA_0,HB_0,HAB_0) - 0.5 H00(A_0,B_0,H_4,C_0,G,HA_0,HB_0,HAB_0) >= 0
0.5 eps_W + 0.5 eps + 0.5 HR(G,inputs_and_H_4,HA_0,HB_0,HAB_0) - 0.5 HR(G,inputs_and_H_4,
    HA_0,HB_0,HAB_0,C_1) - 0.5 H00(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,G,HA_0,HB_0,HAB_0) +
     0.5 H00(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,C_0,C_1,G,HA_0,HB_0,HAB_0) >= 0
0.5 H01(A_0,B_1,H_4,G,HA_0,HB_1,HAB_1) - 0.5 H01(A_0,B_1,H_4,C_0,G,HA_0,HB_1,HAB_1) >= 0
0.5 eps_W + 0.5 eps + 0.5 HR(G,inputs_and_H_4,HA_0,HB_1,HAB_1) - 0.5 HR(G,inputs_and_H_4,
    HA_0,HB_1,HAB_1,C_1) - 0.5 H01(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,G,HA_0,HB_1,HAB_1) +
     0.5 H01(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,C_0,C_1,G,HA_0,HB_1,HAB_1) >= 0
-0.5 H10(B_1) - 0.5 H10(A_1,B_0,C_0) + 0.5 H10(A_0,A_1,B_0,B_1,D,C_0,C_1) == 0
0.5 H10(A_1,B_0,H_4,G,HA_1,HB_0,HAB_1) - 0.5 H10(A_1,B_0,H_4,C_0,G,HA_1,HB_0,HAB_1) >= 0
0.5 H10(A_1,B_0,C_0) - 0.5 H10(A_0,A_1,B_0,B_1,D,C_0,C_1) - 0.5 H10(A_1,B_0,H_4,C_0,G,HA_1,
    HB_0,HAB_1) + 0.5 H10(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,C_0,C_1,G,HA_1,HB_0,HAB_1) >=
     0
0.5 H10(A_1,B_0) - 0.5 H10(A_0,A_1,B_0,B_1,D) - 0.5 H10(A_1,B_0,H_4,G,HA_1,HB_0,HAB_1) + 0.5
     H10(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,G,HA_1,HB_0,HAB_1) >= 0
```

```
0.5 eps_W + eps - 0.5 HR(G,inputs_and_H_4,HA_1) + 0.5 HR(G,inputs_and_H_4,HA_1,HB_0,HAB_1) + 0.5 HR(G,inputs_and_H_4,HA_1,C_1) - 0.5 HR(G,inputs_and_H_4,HA_1,HB_0,HAB_1,C_1) + 0.5 H10(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,G,HA_1) - 0.5 H10(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,C_0,C_1,G,HA_1) - 0.5 H10(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,G,HA_1,HB_0,HAB_1) + 0.5 H10(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,C_0,C_1,G,HA_1,HB_0,HAB_1) >= 0
-H11(B_0) + H11(A_0,A_1,B_0,B_1,D,C_0,C_1) - H11(A_1,B_1,C_0) == 0
H11(A_1,B_1,H_4,G,HA_1,HB_1,HAB_0) - H11(A_1,B_1,H_4,C_0,G,HA_1,HB_1,HAB_0) >= 0
-H11(A_0,A_1,B_0,B_1,D,C_0,C_1) + H11(A_1,B_1,C_0) + H11(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,C_0,C_1,G,HA_1,HB_1,HAB_0) - H11(A_1,B_1,H_4,C_0,G,HA_1,HB_1,HAB_0) >= 0
-H11(A_0,A_1,B_0,B_1,D) + H11(A_1,B_1) + H11(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,G,HA_1,HB_1,HAB_0) - H11(A_1,B_1,H_4,G,HA_1,HB_1,HAB_0) >= 0
0.5 eps_W + eps - 0.5 HR(G,inputs_and_H_4,HB_1) + 0.5 HR(G,inputs_and_H_4,HA_1,HB_1,HAB_0) + 0.5 HR(G,inputs_and_H_4,HB_1,C_1) - 0.5 HR(G,inputs_and_H_4,HA_1,HB_1,HAB_0,C_1) + 0.5 H11(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,G,HB_1) - 0.5 H11(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,C_0,C_1,G,HB_1) - 0.5 H11(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,G,HA_1,HB_1,HAB_0) + 0.5 H11(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,C_0,C_1,G,HA_1,HB_1,HAB_0) >= 0
0.5 eps_W + eps - 0.5 HR(G,inputs_and_H_4,HAB_0) + 0.5 HR(G,inputs_and_H_4,HA_1,HB_1,HAB_0) + 0.5 HR(G,inputs_and_H_4,HAB_0,C_1) - 0.5 HR(G,inputs_and_H_4,HA_1,HB_1,HAB_0,C_1) + 0.5 H11(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,G,HAB_0) - 0.5 H11(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,C_0,C_1,G,HAB_0) - 0.5 H11(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,G,HA_1,HB_1,HAB_0) + 0.5 H11(A_0,A_1,B_0,B_1,D,H_4,inputs_and_H_4,C_0,C_1,G,HA_1,HB_1,HAB_0) >= 0
eps_W + eps + 0.5 H00(H_4,G,HAB_0) - 0.5 H00(A_1,B_1,H_4,C_0,G,HAB_0) - 0.5 H11(H_4,G,HAB_0) + 0.5 H11(A_1,B_1,H_4,C_0,G,HAB_0) >= 0
eps_W + eps - 0.5 H00(H_4,G,HAB_0) + 0.5 H00(A_1,B_1,H_4,C_0,G,HAB_0) + 0.5 H11(H_4,G,HAB_0) - 0.5 H11(A_1,B_1,H_4,C_1,G,HAB_0) >= 0
eps_W + eps - 0.5 H01(H_4,G,HB_1) + 0.5 H01(A_1,B_1,H_4,C_1,G,HB_1) + 0.5 H11(H_4,G,HB_1) - 0.5 H11(A_1,B_1,H_4,C_1,G,HB_1) >= 0
eps_W + eps + 0.5 H01(H_4,G,HB_1) - 0.5 H01(A_1,B_1,H_4,C_1,G,HB_1) - 0.5 H11(H_4,G,HB_1) + 0.5 H11(A_1,B_1,H_4,C_0,G,HB_1) >= 0
eps_W + eps - 0.5 H10(H_4,G,HA_1) + 0.5 H10(A_1,B_0,H_4,C_0,G,HA_1) + 0.5 H11(H_4,G,HA_1) - 0.5 H11(A_1,B_0,H_4,C_0,G,HA_1) >= 0
eps_W + eps + 0.5 H10(H_4,G,HA_1) - 0.5 H10(A_1,B_0,H_4,C_1,G,HA_1) - 0.5 H11(H_4,G,HA_1) + 0.5 H11(A_1,B_0,H_4,C_0,G,HA_1) >= 0

 => 8.5 eps_W + 10 eps - HR(inputs_and_H_4) + HR(G,inputs_and_H_4) >= 1.5
```

A.2 AND-gate without Free-XOR

Citip input:

```
00,01,10,11: A_0,A_1,B_0,B_1,H_4 <- inputs_and_H_4 implicit
00,01,10,11: inputs_and_H_4 <- A_0,A_1,B_0,B_1,H_4 implicit
HR(HA_0,HA_1,HB_0,HB_1 | inputs_and_H_4) >= HR(HA_0) + HR(HA_1) + HR(HB_0) + HR(HB_1)
H00(A_0) = 1
H01(A_0) = 1
H10(A_0) = 1
H11(A_0) = 1
H00(A_1) = 1
H01(A_1) = 1
H10(A_1) = 1
H11(A_1) = 1
H00(B_0) = 1
H01(B_0) = 1
H10(B_0) = 1
H11(B_0) = 1
H00(B_1) = 1
H01(B_1) = 1
H10(B_1) = 1
H11(B_1) = 1
H00(C_0) = 1
H01(C_0) = 1
H10(C_0) = 1
H11(C_0) = 1
H00(C_1) = 1
```

```
H01(C_1) = 1
H10(C_1) = 1
H11(C_1) = 1
00: A_1,B_1,C_1 . A_0,B_0,C_0
00: C_0 <- G,A_0,B_0,H_4,HA_0,HB_0
I00(A_0,A_1,B_0,B_1,C_0,C_1; G,H_4,HA_0,HB_0 | A_0,B_0,C_0) <= 0
I00(A_0,A_1,B_0,B_1; G,H_4,HA_0,HB_0 | A_0,B_0) <= 0
indist R: C_0,C_1,G,inputs_and_H_4,HA_0,HB_0 ; 00: C_0,C_1,G,inputs_and_H_4,HA_0,HB_0 approx
     eps, eps_W,, eps_G, eps_W + eps_H,,
01: A_1,B_0,C_1 . A_0,B_1,C_0
01: C_0 <- G,A_0,B_1,H_4,HA_0,HB_1
I01(A_0,A_1,B_0,B_1,C_0,C_1; G,H_4,HA_0,HB_1 | A_0,B_1,C_0) <= 0
I01(A_0,A_1,B_0,B_1; G,H_4,HA_0,HB_1 | A_0,B_1) <= 0
indist R: C_0,C_1,G,inputs_and_H_4,HA_0,HB_1 ; 01: C_1,C_0,G,inputs_and_H_4,HA_0,HB_1 approx
     eps, eps_W,, eps_G, eps_W + eps_H,,
10: A_0,B_1,C_1 . A_1,B_0,C_0
10: C_0 <- G,A_1,B_0,H_4,HA_1,HB_0
I10(A_0,A_1,B_0,B_1,C_0,C_1; G,H_4,HA_1,HB_0 | A_1,B_0,C_0) <= 0
I10(A_0,A_1,B_0,B_1; G,H_4,HA_1,HB_0 | A_1,B_0) <= 0
indist R: C_0,C_1,G,inputs_and_H_4,HA_1,HB_0 ; 10: C_1,C_0,G,inputs_and_H_4,HA_1,HB_0 approx
     eps, eps_W,, eps_G, eps_W + eps_H,,
11: A_0,B_0,C_1 . A_1,B_1,C_0
11: C_0 <- G,A_1,B_1,H_4,HA_1,HB_1
I11(A_0,A_1,B_0,B_1,C_0,C_1; G,H_4,HA_1,HB_1 | A_1,B_1,C_0) <= 0
I11(A_0,A_1,B_0,B_1; G,H_4,HA_1,HB_1 | A_1,B_1) <= 0
indist R: C_0,C_1,G,inputs_and_H_4,HA_1,HB_1 ; 11: C_1,C_0,G,inputs_and_H_4,HA_1,HB_1 approx
     eps, eps_W,, eps_G, eps_W + eps_H,,
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,G,HA_0,H_4 ; 01: C_0,C_1,A_0,A_1,B_0,B_1,G,HA_0,H_4
    approx 2 eps, 2 eps_W,,,,,, 2 eps_G, 2 eps_H,
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,G,HA_0,H_4 ; 01: C_1,C_0,A_0,A_1,B_0,B_1,G,HA_0,H_4
    approx 2 eps, 2 eps_W,,,,,, 2 eps_G, 2 eps_H,
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,G,HB_0,H_4 ; 10: C_0,C_1,A_0,A_1,B_0,B_1,G,HB_0,H_4
    approx 2 eps, 2 eps_W,,,,,, 2 eps_G, 2 eps_H,
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,G,HB_0,H_4 ; 10: C_1,C_0,A_0,A_1,B_0,B_1,G,HB_0,H_4
    approx 2 eps, 2 eps_W,,,,,, 2 eps_G, 2 eps_H,
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,G,H_4 ; 11: C_0,C_1,A_0,A_1,B_0,B_1,G,H_4 approx 2 eps, 2
     eps_W,,,,,, 2 eps_G, 2 eps_H
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,G,H_4 ; 11: C_1,C_0,A_0,A_1,B_0,B_1,G,H_4 approx 2 eps, 2
     eps_W,,,,,, 2 eps_G, 2 eps_H
indist 01: C_0,C_1,A_0,A_1,B_0,B_1,G,H_4 ; 10: C_0,C_1,A_0,A_1,B_0,B_1,G,H_4 approx 2 eps, 2
     eps_W,,,,,, 2 eps_G, 2 eps_H
indist 01: C_0,C_1,A_0,A_1,B_0,B_1,G,H_4 ; 10: C_1,C_0,A_0,A_1,B_0,B_1,G,H_4 approx 2 eps, 2
     eps_W,,,,,, 2 eps_G, 2 eps_H
indist 01: C_0,C_1,A_0,A_1,B_0,B_1,G,HB_1,H_4 ; 11: C_0,C_1,A_0,A_1,B_0,B_1,G,HB_1,H_4
    approx 2 eps, 2 eps_W,,,,,, 2 eps_G, 2 eps_H,
indist 01: C_0,C_1,A_0,A_1,B_0,B_1,G,HB_1,H_4 ; 11: C_1,C_0,A_0,A_1,B_0,B_1,G,HB_1,H_4
    approx 2 eps, 2 eps_W,,,,,, 2 eps_G, 2 eps_H,
indist 10: C_0,C_1,A_0,A_1,B_0,B_1,G,HA_1,H_4 ; 11: C_0,C_1,A_0,A_1,B_0,B_1,G,HA_1,H_4
    approx 2 eps, 2 eps_W,,,,,, 2 eps_G, 2 eps_H,
indist 10: C_0,C_1,A_0,A_1,B_0,B_1,G,HA_1,H_4 ; 11: C_1,C_0,A_0,A_1,B_0,B_1,G,HA_1,H_4
    approx 2 eps, 2 eps_W,,,,,, 2 eps_G, 2 eps_H,
prove HR(G | inputs_and_H_4) >= c1 + c2 eps_H + c3 eps_G + c4 eps_W + c5 eps
```

Citip output:

```
HR(G | inputs_and_H_4,HA_0,HA_1,HB_0,HB_1,C_0,C_1) >= 0
HR(C_1 | G,inputs_and_H_4,HA_0,HA_1,HB_0,HB_1,C_0) >= 0
IR(G; HA_0) >= 0
IR(G; HA_1 | inputs_and_H_4,HB_1) >= 0
IR(G; HB_0 | HA_1) >= 0
IR(G; HB_1 | inputs_and_H_4,HA_0) >= 0
IR(G; C_0 | inputs_and_H_4,HA_0,HA_1,HB_0,HB_1,C_1) >= 0
IR(G; C_1 | inputs_and_H_4,HA_0,HA_1,HB_0,HB_1) >= 0
IR(inputs_and_H_4; HA_0 | G) >= 0
IR(inputs_and_H_4; HA_1 | HB_1) >= 0
IR(inputs_and_H_4; HB_0 | G,HA_1) >= 0
IR(inputs_and_H_4; HB_1) >= 0
IR(HA_0; HA_1 | G,inputs_and_H_4,HB_1,C_1) >= 0
IR(HA_0; HB_0 | G,inputs_and_H_4,HA_1,C_1) >= 0
```

```
IR(HA_0; HB_1 | inputs_and_H_4) >= 0
IR(HA_1; HB_0) >= 0
IR(HA_1; HB_1) >= 0
IR(HB_0; HB_1 | G,inputs_and_H_4,HA_0,HA_1,C_1) >= 0
IR(C_0; C_1 | G,inputs_and_H_4,HA_0,HA_1,HB_0,HB_1) >= 0
0.333333 I01(A_0; A_1 | B_0,B_1,C_1) >= 0
0.333333 I01(A_0; A_1 | B_0,C_0,C_1) >= 0
0.333333 I01(A_0; A_1 | H_4,C_0,G,HA_0) >= 0
0.333333 I01(A_0; A_1 | B_1,C_1,G,HB_1) >= 0
0.333333 I01(A_0; B_0 | C_0,C_1) >= 0
0.333333 I01(A_0; B_0 | A_1,B_1,H_4,C_1,HB_1) >= 0
0.333333 I01(A_0; H_4 | A_1,B_1,C_1,G,HB_1) >= 0
0.333333 I01(A_0; C_1 | B_1,C_0) >= 0
0.333333 I01(A_0; G | B_1,C_1,HB_1) >= 0
0.333333 I01(A_0; HB_1 | B_1,C_1) >= 0
0.333333 I01(A_1; B_0 | C_1,HB_1) >= 0
0.333333 I01(A_1; B_1 | B_0,C_1) >= 0
0.333333 I01(A_1; B_1 | A_0,B_0,C_0,C_1) >= 0
0.333333 I01(A_1; B_1 | A_0,H_4,C_0,G,HA_0) >= 0
0.333333 I01(A_1; H_4 | C_0,G,HA_0) >= 0
0.333333 I01(A_1; H_4 | A_0,B_0,B_1,C_1,HB_1) >= 0
0.333333 I01(A_1; C_0) >= 0
0.333333 I01(A_1; C_0 | B_0,C_1) >= 0
0.333333 I01(A_1; G | C_0) >= 0
0.333333 I01(A_1; HA_0 | C_0,G) >= 0
0.333333 I01(A_1; HB_1 | A_0,B_0,B_1,C_1) >= 0
0.333333 I01(A_1; HB_1 | A_0,B_1,H_4,C_0,G,HA_0) >= 0
0.333333 I01(B_0; B_1 | A_0,C_0,C_1) >= 0
0.333333 I01(B_0; B_1 | A_1,C_1,HB_1) >= 0
0.333333 I01(B_0; H_4 | A_1,B_1,C_1,HB_1) >= 0
0.333333 I01(B_0; C_0 | C_1) >= 0
0.333333 I01(B_0,inputs_and_H_4; G | A_0,A_1,B_1,H_4,C_1,HB_1) >= 0
0.333333 I01(B_0,inputs_and_H_4; HA_0 | A_0,A_1,B_1,H_4,C_0,G,HB_1) >= 0
0.333333 I01(B_0; HB_1 | C_1) >= 0
0.333333 I01(A_1,B_0,inputs_and_H_4; C_0 | A_0,B_1,H_4,G,HA_0,HB_1) >= 0
0.666667 I01(A_1,B_0,inputs_and_H_4; C_0 | A_0,B_1,H_4,C_1,G,HA_0,HB_1) >= 0
0.333333 I01(A_1,inputs_and_H_4; G | A_0,B_0,B_1,H_4,C_1,HB_1) >= 0
0.333333 I01(A_1,B_0,inputs_and_H_4; HA_0 | A_0,B_1,H_4,C_0,G,HB_1) >= 0
0.333333 I01(C_0; C_1 | B_1) >= 0
0.666667 I01(C_0; C_1 | A_0,B_1,H_4,G,HA_0,HB_1) >= 0
0.666667 I01(C_1; HA_0 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HB_1) >= 0
0.333333 I10(A_0; A_1 | B_0,B_1,C_1) >= 0
0.333333 I10(A_0; A_1 | B_1,H_4,C_1,G,HA_1) >= 0
0.333333 I10(A_0; B_0 | C_1) >= 0
0.333333 I10(A_0; B_0 | A_1,B_1,H_4,G,HA_1) >= 0
0.333333 I10(A_0; B_1 | C_1,HA_1) >= 0
0.333333 I10(A_0; H_4 | A_1) >= 0
0.333333 I10(A_0; H_4 | B_1,C_1,HA_1) >= 0
0.333333 I10(A_0; C_0 | A_1,B_0,B_1,C_1) >= 0
0.333333 I10(A_0,inputs_and_H_4; C_0 | A_1,B_0,B_1,H_4,C_1,G,HA_1,HB_0) >= 0
0.333333 I10(A_0; G | B_1,H_4,C_1,HA_1) >= 0
0.333333 I10(A_0; HA_1 | C_1) >= 0
0.333333 I10(A_0,inputs_and_H_4; HB_0 | A_1,B_0,B_1,H_4,C_1,G,HA_1) >= 0
0.333333 I10(A_1; B_1 | A_0,C_1) >= 0
0.333333 I10(A_1; B_1 | B_0,C_1) >= 0
0.333333 I10(A_1; B_1 | B_0,H_4,C_0,G) >= 0
0.333333 I10(B_0; B_1 | A_0,C_1) >= 0
0.333333 I10(B_0; B_1 | G) >= 0
0.333333 I10(B_0; B_1 | A_0,A_1,C_1,HA_1) >= 0
0.333333 I10(B_0; B_1 | A_1,H_4,C_0,G,HA_1,HB_0) >= 0
0.333333 I10(B_0; H_4 | A_0,A_1) >= 0
0.333333 I10(B_0,inputs_and_H_4; C_1 | A_0,A_1,B_1,H_4,G,HA_1,HB_0) >= 0
0.333333 I10(B_1; H_4 | A_0,A_1,B_0) >= 0
0.333333 I10(B_1; H_4 | B_0,G) >= 0
0.333333 I10(B_1; H_4 | A_0,A_1,B_0,C_1,HA_1) >= 0
0.333333 I10(B_1; C_0 | A_1,B_0) >= 0
0.333333 I10(B_1; C_0 | B_0,H_4,G) >= 0
0.333333 I10(B_1; C_0 | A_1,H_4,G,HA_1) >= 0
```

```
0.333333 I10(B_1; G) >= 0
0.333333 I10(B_1,inputs_and_H_4; G | A_0,A_1,B_0,H_4,C_1,HA_1) >= 0
0.333333 I10(B_1; HA_1 | A_0,A_1,C_1) >= 0
0.333333 I10(B_1; HA_1 | A_1,B_0,H_4,C_0,G) >= 0
0.333333 I10(B_1; HB_0 | A_1,H_4,C_0,G,HA_1) >= 0
0.666667 I10(A_0,B_1,inputs_and_H_4; C_0 | A_1,B_0,H_4,G,HA_1,HB_0) >= 0
0.333333 I10(A_0,B_0,B_1,inputs_and_H_4; G | A_1,H_4,HA_1) >= 0
0.333333 I10(A_0,B_0,B_1,inputs_and_H_4; HA_1 | A_1,H_4) >= 0
0.666667 I10(A_0,B_1,inputs_and_H_4; HB_0 | A_1,B_0,H_4,C_0,G,HA_1) >= 0
0.333333 I10(C_0; C_1 | A_1,B_0,B_1) >= 0
0.333333 I10(C_0; C_1 | A_1,B_0,B_1,H_4,G,HA_1) >= 0
0.333333 I10(C_0; C_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_1,HB_0) >= 0
0.333333 I10(C_1; HB_0 | A_0,A_1,B_1,H_4,G,HA_1) >= 0
0.333333 I10(C_1; HB_0 | A_1,B_0,B_1,H_4,C_0,G,HA_1) >= 0
0.333333 I11(A_0; A_1 | B_0,H_4,C_1,G,HA_1) >= 0
0.333333 I11(A_0; A_1 | B_0,B_1,H_4,C_1,G,HA_1) >= 0
0.666667 I11(A_0; B_0 | C_1,HA_1) >= 0
0.333333 I11(A_0; B_1 | B_0,C_1,HA_1) >= 0
0.333333 I11(A_0; H_4 | B_0,C_1,G,HA_1) >= 0
0.333333 I11(A_0; H_4 | B_0,B_1,C_1,G,HA_1) >= 0
I11(A_0; C_0 | A_1,B_1,H_4,G,HA_1,HB_1) >= 0
0.333333 I11(A_0; G | B_0,C_1,HA_1) >= 0
0.333333 I11(A_0; G | B_0,B_1,C_1,HA_1) >= 0
0.666667 I11(A_0; HA_1 | C_1) >= 0
0.333333 I11(A_0; HB_1 | A_1,B_1,H_4,C_0,G,HA_1) >= 0
0.333333 I11(A_1; B_0 | A_0,B_1,H_4,C_1,G,HB_1) >= 0
0.666667 I11(B_0; B_1 | A_0,C_1) >= 0
0.666667 I11(B_0; H_4 | A_0,B_1,C_1,G,HB_1) >= 0
I11(B_0,inputs_and_H_4; C_0 | A_0,A_1,B_1,H_4,G,HA_1,HB_1) >= 0
0.666667 I11(B_0; G | A_0,B_1,C_1,HB_1) >= 0
0.666667 I11(B_0; HB_1 | A_0,B_1,C_1) >= 0
0.333333 I11(A_0,B_0,inputs_and_H_4; HA_1 | A_1,B_1,H_4,C_0,G,HB_1) >= 0
0.333333 I11(B_0,inputs_and_H_4; HB_1 | A_0,A_1,B_1,H_4,C_0,G,HA_1) >= 0
0.333333 I11(C_0; C_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HB_1) >= 0
0.333333 I11(C_0; C_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_1,HB_1) >= 0
0.333333 I11(C_1; HA_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HB_1) >= 0
0.333333 I11(C_1; HB_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_1) >= 0
-HR(inputs_and_H_4) - HR(HA_0) - HR(HA_1) - HR(HB_0) - HR(HB_1) + HR(inputs_and_H_4,HA_0,
    HA_1,HB_0,HB_1) >= 0
-0.333333 H01(A_1) == -0.333333
0.333333 H10(A_1) == 0.333333
0.333333 H01(B_1) == 0.333333
-0.333333 H10(B_1) == -0.333333
0.666667 H01(C_1) == 0.666667
0.666667 H10(C_1) == 0.666667
0.666667 H11(C_1) == 0.666667
-H01(A_0,B_1,C_0) - H01(A_1,B_0,C_1) + H01(A_0,A_1,B_0,B_1,C_0,C_1) == 0
H01(A_0,B_1,H_4,G,HA_0,HB_1) - H01(A_0,B_1,H_4,C_0,G,HA_0,HB_1) >= 0
0.666667 H01(A_0,B_1,C_0) - 0.666667 H01(A_0,A_1,B_0,B_1,C_0,C_1) - 0.666667 H01(A_0,B_1,H_4
    ,C_0,G,HA_0,HB_1) + 0.666667 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_0,HB_1)
     >= 0
0.666667 eps_W + 1.33333 eps - 0.666667 HR(G,inputs_and_H_4,HB_1) + 0.666667 HR(G,
    inputs_and_H_4,HA_0,HB_1) + 0.666667 HR(G,inputs_and_H_4,HB_1,C_1) - 0.666667 HR(G,
    inputs_and_H_4,HA_0,HB_1,C_1) + 0.666667 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HB_1)
    - 0.666667 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HB_1) - 0.666667 H01(A_0,A_1,B_0
    ,B_1,H_4,inputs_and_H_4,G,HA_0,HB_1) + 0.666667 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,
    C_0,G,HA_0,HB_1) >= 0
0.333333 eps_W + 0.333333 eps + 0.333333 HR(G,inputs_and_H_4,HA_0,HB_1) - 0.333333 HR(G,
    inputs_and_H_4,HA_0,HB_1,C_1) - 0.333333 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_0,
    HB_1) + 0.333333 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_0,HB_1) >= 0
-H10(A_1,B_0,C_0) - H10(A_0,B_1,C_1) + H10(A_0,A_1,B_0,B_1,C_0,C_1) == 0
H10(A_1,B_0,H_4,G,HA_1,HB_0) - H10(A_1,B_0,H_4,C_0,G,HA_1,HB_0) >= 0
0.666667 H10(A_1,B_0,C_0) - 0.666667 H10(A_0,A_1,B_0,B_1,C_0,C_1) - 0.666667 H10(A_1,B_0,H_4
    ,C_0,G,HA_1,HB_0) + 0.666667 H10(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_1,HB_0)
     >= 0
0.333333 H10(A_1,B_0) - 0.333333 H10(A_0,A_1,B_0,B_1) - 0.333333 H10(A_1,B_0,H_4,G,HA_1,HB_0
    ) + 0.333333 H10(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_1,HB_0) >= 0
0.666667 eps_W + 1.33333 eps - 0.666667 HR(G,inputs_and_H_4,HA_1) + 0.666667 HR(G,
```

```
    inputs_and_H_4,HA_1,HB_0) + 0.666667 HR(G,inputs_and_H_4,HA_1,C_1) - 0.666667 HR(G,
    inputs_and_H_4,HA_1,HB_0,C_1) + 0.666667 H10(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_1)
    - 0.666667 H10(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_1) - 0.666667 H10(A_0,A_1,B_0
    ,B_1,H_4,inputs_and_H_4,G,HA_1,HB_0) + 0.666667 H10(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,
    C_0,G,HA_1,HB_0) >= 0
138 0.333333 eps_W + 0.333333 eps + 0.333333 HR(G,inputs_and_H_4,HA_1,HB_0) - 0.333333 HR(G,
    inputs_and_H_4,HA_1,HB_0,C_1) - 0.333333 H10(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_1,
    HB_0) + 0.333333 H10(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_1,HB_0) >= 0
139 -0.666667 H11(A_1,B_1,C_0) - 0.666667 H11(A_0,B_0,C_1) + 0.666667 H11(A_0,A_1,B_0,B_1,C_0,
    C_1) == 0
140 H11(A_1,B_1,H_4,G,HA_1,HB_1) - H11(A_1,B_1,H_4,C_0,G,HA_1,HB_1) >= 0
141 0.666667 H11(A_1,B_1,C_0) - 0.666667 H11(A_0,A_1,B_0,B_1,C_0,C_1) - 0.666667 H11(A_1,B_1,H_4
    ,C_0,G,HA_1,HB_1) + 0.666667 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_1,HB_1)
     >= 0
142 0.333333 eps_W + 0.666667 eps - 0.333333 HR(G,inputs_and_H_4,HA_1) + 0.333333 HR(G,
    inputs_and_H_4,HA_1,HB_1) + 0.333333 HR(G,inputs_and_H_4,HA_1,C_1) - 0.333333 HR(G,
    inputs_and_H_4,HA_1,HB_1,C_1) + 0.333333 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_1)
    - 0.333333 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_1) - 0.333333 H11(A_0,A_1,B_0
    ,B_1,H_4,inputs_and_H_4,G,HA_1,HB_1) + 0.333333 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,
    C_0,G,HA_1,HB_1) >= 0
143 0.333333 eps_W + 0.666667 eps - 0.333333 HR(G,inputs_and_H_4,HB_1) + 0.333333 HR(G,
    inputs_and_H_4,HA_1,HB_1) + 0.333333 HR(G,inputs_and_H_4,HB_1,C_1) - 0.333333 HR(G,
    inputs_and_H_4,HA_1,HB_1,C_1) + 0.333333 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HB_1)
    - 0.333333 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HB_1) - 0.333333 H11(A_0,A_1,B_0
    ,B_1,H_4,inputs_and_H_4,G,HA_1,HB_1) + 0.333333 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,
    C_0,G,HA_1,HB_1) >= 0
144 0.333333 eps_W + 0.333333 eps + 0.333333 HR(G,inputs_and_H_4,HA_1,HB_1) - 0.333333 HR(G,
    inputs_and_H_4,HA_1,HB_1,C_1) - 0.333333 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_1,
    HB_1) + 0.333333 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_1,HB_1) >= 0
145 0.666667 eps_W + 0.666667 eps + 0.333333 H01(A_1,B_0,C_1) - 0.333333 H10(A_1,B_0,C_1) >= 0
146 0.666667 eps_W + 0.666667 eps - 0.333333 H01(A_0,B_1,C_1) + 0.333333 H10(A_0,B_1,C_1) >= 0
147 0.666667 eps_W + 0.666667 eps + 0.333333 H01(H_4,G,HB_1) - 0.333333 H01(A_0,B_0,B_1,H_4,C_1,
    G,HB_1) - 0.333333 H11(H_4,G,HB_1) + 0.333333 H11(A_0,B_0,B_1,H_4,C_1,G,HB_1) >= 0
148 0.666667 eps_W + 0.666667 eps - 0.333333 H01(H_4,G,HB_1) + 0.333333 H01(A_0,B_1,H_4,C_0,G,
    HB_1) + 0.333333 H11(H_4,G,HB_1) - 0.333333 H11(A_0,B_1,H_4,C_1,G,HB_1) >= 0
149 0.666667 eps_W + 0.666667 eps + 0.333333 H01(H_4,G,HB_1) - 0.333333 H01(A_1,B_1,H_4,C_1,G,
    HB_1) - 0.333333 H11(H_4,G,HB_1) + 0.333333 H11(A_1,B_1,H_4,C_0,G,HB_1) >= 0
150 0.666667 eps_W + 0.666667 eps - 0.333333 H01(H_4,G,HB_1) + 0.333333 H01(A_0,A_1,B_1,H_4,C_0,
    G,HB_1) + 0.333333 H11(H_4,G,HB_1) - 0.333333 H11(A_0,A_1,B_1,H_4,C_1,G,HB_1) >= 0
151 0.666667 eps_W + 0.666667 eps + 0.333333 H10(H_4,G,HA_1) - 0.333333 H10(A_0,A_1,B_0,H_4,C_1,
    G,HA_1) - 0.333333 H11(H_4,G,HA_1) + 0.333333 H11(A_0,A_1,B_0,H_4,C_1,G,HA_1) >= 0
152 0.666667 eps_W + 0.666667 eps - 0.333333 H10(H_4,G,HA_1) + 0.333333 H10(A_1,B_0,H_4,C_0,G,
    HA_1) + 0.333333 H11(H_4,G,HA_1) - 0.333333 H11(A_1,B_0,H_4,C_1,G,HA_1) >= 0
153 0.666667 eps_W + 0.666667 eps + 0.333333 H10(H_4,G,HA_1) - 0.333333 H10(A_1,B_1,H_4,C_1,G,
    HA_1) - 0.333333 H11(H_4,G,HA_1) + 0.333333 H11(A_1,B_1,H_4,C_0,G,HA_1) >= 0
154 0.666667 eps_W + 0.666667 eps - 0.333333 H10(H_4,G,HA_1) + 0.333333 H10(A_1,B_0,B_1,H_4,C_0,
    G,HA_1) + 0.333333 H11(H_4,G,HA_1) - 0.333333 H11(A_1,B_0,B_1,H_4,C_1,G,HA_1) >= 0
155
156  => 9.66667 eps_W + 11.6667 eps - HR(inputs_and_H_4) + HR(G,inputs_and_H_4) >= 2
```

A.3 XOR-gate without Free-XOR

Citip input:

```
1  00,01,10,11: A_0,A_1,B_0,B_1,H_4 <- inputs_and_H_4 implicit
2  00,01,10,11: inputs_and_H_4 <- A_0,A_1,B_0,B_1,H_4 implicit
3  HR(HA_0,HA_1,HB_0,HB_1 | inputs_and_H_4) >= HR(HA_0) + HR(HA_1) + HR(HB_0) + HR(HB_1)
4  H00(A_0) = 1
5  H01(A_0) = 1
6  H10(A_0) = 1
7  H11(A_0) = 1
8  H00(A_1) = 1
9  H01(A_1) = 1
10 H10(A_1) = 1
11 H11(A_1) = 1
12 H00(B_0) = 1
13 H01(B_0) = 1
```

```
H10(B_0) = 1
H11(B_0) = 1
H00(B_1) = 1
H01(B_1) = 1
H10(B_1) = 1
H11(B_1) = 1
H00(C_0) = 1
H01(C_0) = 1
H10(C_0) = 1
H11(C_0) = 1
H00(C_1) = 1
H01(C_1) = 1
H10(C_1) = 1
H11(C_1) = 1
00: A_1,B_1,C_1 . A_0,B_0,C_0
00: C_0 <- G,A_0,B_0,H_4,HA_0,HB_0
I00(A_0,A_1,B_0,B_1,C_0,C_1; G,H_4,HA_0,HB_0 | A_0,B_0,C_0) <= 0
I00(A_0,A_1,B_0,B_1; G,H_4,HA_0,HB_0 | A_0,B_0) <= 0
indist R: C_0,C_1,G,inputs_and_H_4,HA_0,HB_0 ; 00: C_0,C_1,G,inputs_and_H_4,HA_0,HB_0 approx
     eps, eps_W,, eps_G, eps_W + eps_H,,
01: A_1,B_0,C_1 . A_0,B_1,C_0
01: C_0 <- G,A_0,B_1,H_4,HA_0,HB_1
I01(A_0,A_1,B_0,B_1,C_0,C_1; G,H_4,HA_0,HB_1 | A_0,B_1,C_0) <= 0
I01(A_0,A_1,B_0,B_1; G,H_4,HA_0,HB_1 | A_0,B_1) <= 0
indist R: C_0,C_1,G,inputs_and_H_4,HA_0,HB_1 ; 01: C_1,C_0,G,inputs_and_H_4,HA_0,HB_1 approx
     eps, eps_W,, eps_G, eps_W + eps_H,,
10: A_0,B_1,C_1 . A_1,B_0,C_0
10: C_0 <- G,A_1,B_0,H_4,HA_1,HB_0
I10(A_0,A_1,B_0,B_1,C_0,C_1; G,H_4,HA_1,HB_0 | A_1,B_0,C_0) <= 0
I10(A_0,A_1,B_0,B_1; G,H_4,HA_1,HB_0 | A_1,B_0) <= 0
indist R: C_0,C_1,G,inputs_and_H_4,HA_1,HB_0 ; 10: C_1,C_0,G,inputs_and_H_4,HA_1,HB_0 approx
     eps, eps_W,, eps_G, eps_W + eps_H,,
11: A_0,B_0,C_1 . A_1,B_1,C_0
11: C_0 <- G,A_1,B_1,H_4,HA_1,HB_1
I11(A_0,A_1,B_0,B_1,C_0,C_1; G,H_4,HA_1,HB_1 | A_1,B_1,C_0) <= 0
I11(A_0,A_1,B_0,B_1; G,H_4,HA_1,HB_1 | A_1,B_1) <= 0
indist R: C_0,C_1,G,inputs_and_H_4,HA_1,HB_1 ; 11: C_0,C_1,G,inputs_and_H_4,HA_1,HB_1 approx
     eps, eps_W,, eps_G, eps_W + eps_H,,
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,G,HA_0,H_4 ; 01: C_1,C_0,A_0,A_1,B_0,B_1,G,HA_0,H_4
    approx 2 eps, 2 eps_W,,,,,, 2 eps_G, 2 eps_H,
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,G,HB_0,H_4 ; 10: C_1,C_0,A_0,A_1,B_0,B_1,G,HB_0,H_4
    approx 2 eps, 2 eps_W,,,,,, 2 eps_G, 2 eps_H,
indist 00: C_0,C_1,A_0,A_1,B_0,B_1,G,H_4 ; 11: C_0,C_1,A_0,A_1,B_0,B_1,G,H_4 approx 2 eps, 2
     eps_W,,,,,, 2 eps_G, 2 eps_H
indist 01: C_0,C_1,A_0,A_1,B_0,B_1,G,H_4 ; 10: C_0,C_1,A_0,A_1,B_0,B_1,G,H_4 approx 2 eps, 2
     eps_W,,,,,, 2 eps_G, 2 eps_H
indist 01: C_0,C_1,A_0,A_1,B_0,B_1,G,HB_1,H_4 ; 11: C_1,C_0,A_0,A_1,B_0,B_1,G,HB_1,H_4
    approx 2 eps, 2 eps_W,,,,,, 2 eps_G, 2 eps_H,
indist 10: C_0,C_1,A_0,A_1,B_0,B_1,G,HA_1,H_4 ; 11: C_1,C_0,A_0,A_1,B_0,B_1,G,HA_1,H_4
    approx 2 eps, 2 eps_W,,,,,, 2 eps_G, 2 eps_H,
prove HR(G | inputs_and_H_4) >= c1 + c2 eps_H + c3 eps_G + c4 eps_W + c5 eps
```

Citip output:

```
HR(G | inputs_and_H_4,HA_0,HA_1,HB_0,HB_1,C_0,C_1) >= 0
0.555556 HR(C_0 | G,inputs_and_H_4,HA_0,HA_1,HB_0,HB_1,C_1) >= 0
0.555556 HR(C_1 | G,inputs_and_H_4,HA_0,HA_1,HB_0,HB_1,C_0) >= 0
0.833333 IR(G; HA_0) >= 0
0.0833333 IR(G; HA_0 | inputs_and_H_4) >= 0
0.0833333 IR(G; HA_0 | inputs_and_H_4,C_1) >= 0
0.0833333 IR(G; HA_1 | inputs_and_H_4,HA_0) >= 0
0.277778 IR(G; HA_1 | inputs_and_H_4,HB_0) >= 0
0.333333 IR(G; HA_1 | inputs_and_H_4,HB_1) >= 0
0.111111 IR(G; HA_1 | inputs_and_H_4,HB_0,HB_1) >= 0
0.0833333 IR(G; HA_1 | HA_0,C_0) >= 0
0.0833333 IR(G; HA_1 | C_1) >= 0
0.0277778 IR(G; HA_1 | inputs_and_H_4,C_0,C_1) >= 0
0.111111 IR(G; HB_0 | inputs_and_H_4) >= 0
0.277778 IR(G; HB_0 | inputs_and_H_4,HA_0) >= 0
```

```
0.166667 IR(G; HB_0 | HA_1) >= 0
0.444444 IR(G; HB_0 | HB_1) >= 0
0.611111 IR(G; HB_1 | inputs_and_H_4) >= 0
0.277778 IR(G; HB_1 | HA_0) >= 0
0.111111 IR(G; HB_1 | inputs_and_H_4,HA_1,HB_0,C_1) >= 0
IR(G; C_0 | inputs_and_H_4,HA_0,HA_1,HB_0,HB_1,C_1) >= 0
IR(G; C_1 | inputs_and_H_4,HA_0,HA_1,HB_0,HB_1) >= 0
0.222222 IR(inputs_and_H_4; HA_0 | G) >= 0
0.0833333 IR(inputs_and_H_4; HA_0 | HA_1) >= 0
0.444444 IR(inputs_and_H_4; HA_0 | G,HB_1) >= 0
0.166667 IR(inputs_and_H_4; HA_0 | G,HB_0,HB_1) >= 0
0.0833333 IR(inputs_and_H_4; HA_0 | C_1) >= 0
0.25 IR(inputs_and_H_4; HA_1) >= 0
0.333333 IR(inputs_and_H_4; HA_1 | HB_1) >= 0
0.222222 IR(inputs_and_H_4; HA_1 | HB_0,HB_1) >= 0
0.0833333 IR(inputs_and_H_4; HA_1 | G,HA_0,HB_0,C_0) >= 0
0.0833333 IR(inputs_and_H_4; HA_1 | G,HA_0,C_1) >= 0
0.0277778 IR(inputs_and_H_4; HA_1 | C_0,C_1) >= 0
0.277778 IR(inputs_and_H_4; HB_0) >= 0
0.277778 IR(inputs_and_H_4; HB_0 | G) >= 0
0.166667 IR(inputs_and_H_4; HB_0 | G,HA_1) >= 0
0.111111 IR(inputs_and_H_4; HB_0 | HB_1) >= 0
0.166667 IR(inputs_and_H_4; HB_0 | G,HA_0,HB_1,C_1) >= 0
0.722222 IR(inputs_and_H_4; HB_1) >= 0
0.277778 IR(inputs_and_H_4; HB_1 | G,HB_0) >= 0
0.0833333 IR(HA_0; HA_1) >= 0
0.0833333 IR(HA_0; HA_1 | C_0) >= 0
0.0833333 IR(HA_0; HA_1 | G,inputs_and_H_4,C_0) >= 0
0.194444 IR(HA_0; HA_1 | G,inputs_and_H_4,HB_0,HB_1,C_0) >= 0
0.0833333 IR(HA_0; HA_1 | G,C_1) >= 0
0.333333 IR(HA_0; HA_1 | G,inputs_and_H_4,HB_0,C_1) >= 0
0.0277778 IR(HA_0; HA_1 | G,inputs_and_H_4,HB_1,C_1) >= 0
0.0277778 IR(HA_0; HA_1 | G,inputs_and_H_4,HB_0,C_0,C_1) >= 0
0.0277778 IR(HA_0; HA_1 | G,inputs_and_H_4,HB_1,C_0,C_1) >= 0
0.0555556 IR(HA_0; HA_1 | G,inputs_and_H_4,HB_0,HB_1,C_0,C_1) >= 0
0.333333 IR(HA_0; HB_0 | G) >= 0
0.277778 IR(HA_0; HB_0 | inputs_and_H_4) >= 0
0.25 IR(HA_0; HB_0 | G,inputs_and_H_4,C_1) >= 0
0.111111 IR(HA_0; HB_0 | G,inputs_and_H_4,HA_1,HB_1,C_1) >= 0
0.0277778 IR(HA_0; HB_0 | G,inputs_and_H_4,HB_1,C_0,C_1) >= 0
0.277778 IR(HA_0; HB_1) >= 0
0.333333 IR(HA_0; HB_1 | G,HB_0) >= 0
0.0833333 IR(HA_0; HB_1 | G,inputs_and_H_4,C_0) >= 0
0.0277778 IR(HA_0; HB_1 | G,inputs_and_H_4,HB_0,C_0) >= 0
0.25 IR(HA_0; HB_1 | G,inputs_and_H_4,HA_1,HB_0,C_0) >= 0
0.0277778 IR(HA_0; HB_1 | G,inputs_and_H_4,HA_1,C_0,C_1) >= 0
0.0833333 IR(HA_0; C_0 | G,inputs_and_H_4,HA_1) >= 0
0.0833333 IR(HA_0; C_1) >= 0
0.388889 IR(HA_1; HB_0) >= 0
0.166667 IR(HA_1; HB_0 | inputs_and_H_4) >= 0
0.0833333 IR(HA_1; HB_0 | G,HA_0,C_0) >= 0
0.166667 IR(HA_1; HB_0 | G,inputs_and_H_4,HA_0,C_0) >= 0
0.166667 IR(HA_1; HB_0 | G,inputs_and_H_4,HB_1,C_0) >= 0
0.0277778 IR(HA_1; HB_0 | G,inputs_and_H_4,HA_0,C_0,C_1) >= 0
0.333333 IR(HA_1; HB_1) >= 0
0.222222 IR(HA_1; HB_1 | HB_0) >= 0
0.0833333 IR(HA_1; HB_1 | G,inputs_and_H_4,HA_0,C_1) >= 0
0.333333 IR(HA_1; HB_1 | G,inputs_and_H_4,HA_0,HB_0,C_1) >= 0
0.0277778 IR(HA_1; HB_1 | G,inputs_and_H_4,HB_0,C_0,C_1) >= 0
0.0833333 IR(HA_1; C_0) >= 0
0.0277778 IR(HA_1; C_0 | C_1) >= 0
0.111111 IR(HA_1; C_1) >= 0
0.555556 IR(HB_0; HB_1) >= 0
0.166667 IR(HB_0; HB_1 | G,inputs_and_H_4,HA_1,C_0) >= 0
0.166667 IR(HB_0; HB_1 | G,inputs_and_H_4,HA_0,C_1) >= 0
0.0277778 IR(HB_0; HB_1 | G,inputs_and_H_4,HA_0,C_0,C_1) >= 0
0.0277778 IR(HB_0; HB_1 | G,inputs_and_H_4,HA_1,C_0,C_1) >= 0
0.0555556 IR(HB_0; HB_1 | G,inputs_and_H_4,HA_0,HA_1,C_0,C_1) >= 0
```

```
0.111111 IR(HB_0; C_0 | G,inputs_and_H_4,HA_1,HB_1) >= 0
0.166667 IR(HB_0; C_1 | G,HA_0,HB_1) >= 0
0.166667 IR(HB_1; C_0 | G,inputs_and_H_4,HA_0,HB_0) >= 0
0.111111 IR(HB_1; C_1 | inputs_and_H_4,HA_1,HB_0) >= 0
0.111111 IR(C_0; C_1 | G,inputs_and_H_4) >= 0
IR(C_0; C_1 | G,inputs_and_H_4,HA_0,HA_1,HB_0,HB_1) >= 0
0.0246914 IO0(A_0; A_1 | B_0) >= 0
0.141975 IO0(A_0; A_1 | B_0,B_1) >= 0
0.166667 IO0(A_0; A_1 | C_1) >= 0
0.0462963 IO0(A_0; A_1 | B_1,H_4,C_1,G) >= 0
0.037037 IO0(A_0; A_1 | B_0,B_1,H_4,C_0,C_1,G,HA_0) >= 0
0.0833333 IO0(A_0; B_1 | B_0) >= 0
0.0833333 IO0(A_0; B_1 | B_0,C_0) >= 0
0.037037 IO0(A_0; B_1 | A_1,C_1) >= 0
0.0462963 IO0(A_0; B_1 | C_0,C_1) >= 0
0.0833333 IO0(A_0; B_1 | A_1,B_0,C_1,HB_0) >= 0
0.0833333 IO0(A_0; H_4 | A_1,B_0,B_1,C_1,HB_0) >= 0
0.0833333 IO0(A_0; C_1 | A_1,B_0,B_1) >= 0
0.0833333 IO0(A_0; HB_0 | A_1,B_0,C_1) >= 0
0.0833333 IO0(A_1; B_0) >= 0
0.191358 IO0(A_1; B_0 | A_0,C_1) >= 0
0.00925926 IO0(A_1; B_0 | A_0,H_4,G) >= 0
0.037037 IO0(A_1; B_0 | B_1,H_4,C_0,C_1,G,HA_0) >= 0
0.0123457 IO0(A_1; B_0 | A_0,B_1,H_4,C_0,G,HA_0,HB_0) >= 0
0.0123457 IO0(A_1; B_1 | A_0) >= 0
0.058642 IO0(A_1; B_1 | B_0) >= 0
0.0462963 IO0(A_1; B_1 | A_0,B_0,C_0,C_1) >= 0
0.037037 IO0(A_1; B_1 | C_1,HA_0) >= 0
0.0123457 IO0(A_1; B_1 | A_0,C_0,G,HA_0,HB_0) >= 0
0.0709876 IO0(A_1; H_4 | A_0,B_0,B_1) >= 0
0.0462963 IO0(A_1; H_4 | B_1,C_1,G) >= 0
0.037037 IO0(A_1; H_4 | B_1,C_0,C_1,G,HA_0) >= 0
0.0123457 IO0(A_1; H_4 | A_0,B_1,C_0,G,HA_0,HB_0) >= 0
0.0123457 IO0(A_1; C_0 | A_0,B_0) >= 0
0.0123457 IO0(A_1; C_0 | A_0,B_1) >= 0
0.108025 IO0(A_1; C_0 | A_0,B_0,B_1) >= 0
0.0462963 IO0(A_1; C_0 | A_0,B_0,C_1) >= 0
0.037037 IO0(A_1; C_0 | B_1,C_1,HA_0) >= 0
0.361111 IO0(A_1,inputs_and_H_4; C_0 | A_0,B_0,B_1,H_4,G,HA_0,HB_0) >= 0
0.175926 IO0(A_1,inputs_and_H_4; C_0 | A_0,B_0,B_1,H_4,C_1,G,HA_0,HB_0) >= 0
0.0462963 IO0(A_1; G | B_1,C_1) >= 0
0.0123457 IO0(A_1; G | A_0,C_0,HA_0) >= 0
0.037037 IO0(A_1; G | B_1,C_0,C_1,HA_0) >= 0
0.0123457 IO0(A_1; HA_0 | A_0,C_0) >= 0
0.037037 IO0(A_1; HA_0 | C_1) >= 0
0.0123457 IO0(A_1; HB_0 | A_0,C_0,G,HA_0) >= 0
0.0833333 IO0(B_0; B_1 | C_0) >= 0
0.0123457 IO0(B_0; B_1 | A_0,A_1,C_0) >= 0
0.166667 IO0(B_0; B_1 | A_1,C_1) >= 0
0.0740741 IO0(B_0; B_1 | A_0,A_1,C_1) >= 0
0.0216049 IO0(B_0; B_1 | A_0,C_0,C_1) >= 0
0.0277778 IO0(B_0; B_1 | A_0,H_4,G) >= 0
0.00925926 IO0(B_0; B_1 | A_0,A_1,H_4,G) >= 0
0.00925926 IO0(B_0; B_1 | A_0,H_4,C_0,G) >= 0
0.0123457 IO0(B_0; B_1 | A_0,H_4,C_0,G,HA_0) >= 0
0.0462963 IO0(B_0; H_4 | A_0) >= 0
0.037037 IO0(B_0; H_4 | A_0,A_1,B_1,C_1,G) >= 0
0.0123457 IO0(B_0; C_1 | A_0,A_1) >= 0
0.0246914 IO0(B_0; C_1 | A_0,B_1,C_0) >= 0
0.00925926 IO0(B_0; C_1 | A_0,B_1,H_4,C_0,G) >= 0
0.0277778 IO0(B_0; C_1 | A_0,B_1,H_4,G,HA_0) >= 0
0.00925926 IO0(B_0,inputs_and_H_4; C_1 | A_0,A_1,B_1,H_4,G,HA_0) >= 0
0.037037 IO0(B_0; G | A_0,A_1,B_1,C_1) >= 0
0.0277778 IO0(B_0; HA_0 | A_0,B_1,H_4,G) >= 0
0.0709877 IO0(B_1; H_4 | A_0,B_0) >= 0
0.0123457 IO0(B_1; H_4 | A_0,C_0) >= 0
0.0833333 IO0(B_1; H_4 | A_1,B_0,C_1,HB_0) >= 0
0.0833333 IO0(B_1; C_0) >= 0
```

```
0.0493827 I00(B_1; C_0 | A_0,B_0) >= 0
0.0462963 I00(B_1; C_0 | C_1) >= 0
0.00925926 I00(B_1; C_0 | A_0,H_4,C_1,G) >= 0
0.037037 I00(B_1; C_0 | A_0,B_0,H_4,C_1,G,HA_0) >= 0
0.527778 I00(B_1; C_0 | A_0,B_0,H_4,G,HA_0,HB_0) >= 0
0.0123457 I00(B_1; G | A_0,H_4,C_0,HA_0) >= 0
0.0123457 I00(B_1; HA_0 | A_0,H_4,C_0) >= 0
0.00925925 I00(B_1; HA_0 | A_0,B_0,H_4,C_0,C_1,G,HB_0) >= 0
0.0833333 I00(B_1; HB_0 | A_1,B_0,C_1) >= 0
0.00925926 I00(B_1; HB_0 | A_0,B_0,H_4,C_0,C_1,G) >= 0
0.0123457 I00(B_1; HB_0 | A_0,B_0,H_4,C_0,G,HA_0) >= 0
0.0246914 I00(A_1,B_1,inputs_and_H_4; C_0 | A_0,B_0,H_4,HA_0) >= 0
0.0462963 I00(A_1,B_0,B_1,inputs_and_H_4; G | A_0,H_4) >= 0
0.0833333 I00(A_0,B_1,inputs_and_H_4; G | A_1,B_0,H_4,C_1,HB_0) >= 0
0.0246914 I00(A_1,B_1,inputs_and_H_4; G | A_0,B_0,H_4,C_0,HA_0,HB_0) >= 0
0.0246914 I00(A_1,B_1,inputs_and_H_4; HA_0 | A_0,B_0,H_4) >= 0
0.0555556 I00(A_1,B_1,inputs_and_H_4; HA_0 | A_0,B_0,H_4,C_0,G) >= 0
0.037037 I00(A_1,B_1,inputs_and_H_4; HA_0 | A_0,B_0,H_4,C_1,G) >= 0
0.0833333 I00(A_1,B_1,inputs_and_H_4; HA_0 | A_0,B_0,H_4,C_0,G,HB_0) >= 0
0.0246914 I00(A_1,B_1,inputs_and_H_4; HB_0 | A_0,B_0,H_4,C_0,HA_0) >= 0
0.0185185 I00(A_1,B_1,inputs_and_H_4; HB_0 | A_0,B_0,H_4,C_0,G,HA_0) >= 0
0.00925926 I00(A_1,inputs_and_H_4; HB_0 | A_0,B_0,B_1,H_4,C_1,G,HA_0) >= 0
0.101852 I00(A_1,B_1,inputs_and_H_4; HB_0 | A_0,B_0,H_4,C_0,C_1,G,HA_0) >= 0
0.0246914 I00(C_0; C_1 | A_0) >= 0
0.0216049 I00(C_0; C_1 | A_0,B_0) >= 0
0.12037 I00(C_0; C_1 | A_0,A_1,B_0,B_1) >= 0
0.00925926 I00(C_0; C_1 | A_0,H_4,G) >= 0
0.037037 I00(C_0; C_1 | A_0,B_0,H_4,G) >= 0
0.037037 I00(C_0; C_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_0) >= 0
0.166667 I00(C_0; C_1 | A_0,B_0,B_1,H_4,G,HA_0,HB_0) >= 0
0.00925926 I00(C_1; HA_0 | A_0,A_1,B_1,H_4,G) >= 0
0.0462963 I00(C_1; HA_0 | A_0,B_0,H_4,C_0,G) >= 0
0.0833333 I00(C_1; HA_0 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HB_0) >= 0
0.0555556 I00(C_1; HA_0 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HB_0) >= 0
0.111111 I00(C_1; HB_0 | A_0,B_0,H_4,C_0,G,HA_0) >= 0
0.0833333 I01(A_0; A_1 | B_0,B_1,C_1) >= 0
0.166667 I01(A_0; A_1 | B_0,C_0,C_1) >= 0
0.0833333 I01(A_0; A_1 | H_4,C_0,G,HA_0) >= 0
0.0833333 I01(A_0; A_1 | B_1,C_1,G,HB_1) >= 0
0.166667 I01(A_0; B_0 | C_0,C_1) >= 0
0.0833333 I01(A_0; B_0 | A_1,B_1,H_4,G) >= 0
0.0833333 I01(A_0; B_0 | A_1,B_1,H_4,C_1,HB_1) >= 0
0.0833333 I01(A_0; H_4 | A_1,B_1,C_1,G,HB_1) >= 0
0.0833333 I01(A_0; C_1 | B_1,C_0) >= 0
0.0833333 I01(A_0; G | B_1,C_1,HB_1) >= 0
0.0833333 I01(A_0; HB_1 | B_1,C_1) >= 0
0.0833333 I01(A_1; B_0 | H_4,G) >= 0
0.0833333 I01(A_1; B_0 | C_1,HB_1) >= 0
0.0833333 I01(A_1; B_1 | B_0,C_1) >= 0
0.166667 I01(A_1; B_1 | A_0,B_0,C_0,C_1) >= 0
0.0833333 I01(A_1; B_1 | A_0,H_4,C_0,G,HA_0) >= 0
0.0833333 I01(A_1; H_4 | A_0,B_0,B_1,C_1) >= 0
0.0833333 I01(A_1; H_4 | C_0,G,HA_0) >= 0
0.0833333 I01(A_1; C_0) >= 0
0.166667 I01(A_1; C_0 | B_0,C_1) >= 0
0.0833333 I01(A_1; G | C_0) >= 0
0.0833333 I01(A_1,inputs_and_H_4; G | A_0,B_0,B_1,H_4,C_1) >= 0
0.0833333 I01(A_1; HA_0 | C_0,G) >= 0
0.0555556 I01(A_1; HA_0 | A_0,B_1,H_4,C_0,G) >= 0
0.0277778 I01(A_1; HA_0 | A_0,B_0,H_4,C_1,G) >= 0
0.138889 I01(A_1; HB_1 | A_0,B_1,H_4,C_0,G,HA_0) >= 0
0.0833333 I01(B_0; B_1 | A_0,C_1) >= 0
0.166667 I01(B_0; B_1 | A_0,C_0,C_1) >= 0
0.0833333 I01(B_0; B_1 | A_1,H_4,G) >= 0
0.0833333 I01(B_0; B_1 | A_1,C_1,HB_1) >= 0
0.0833333 I01(B_0; H_4 | G) >= 0
0.0833333 I01(B_0; H_4 | A_1,B_1,C_1,HB_1) >= 0
0.166667 I01(B_0; C_0 | C_1) >= 0
```

```
0.0833333 I01(B_0; G) >= 0
0.0833333 I01(B_0,inputs_and_H_4; HA_0 | A_0,A_1,B_1,H_4,C_1,HB_1) >= 0
0.0555556 I01(B_0,inputs_and_H_4; HA_0 | A_0,A_1,B_1,H_4,C_0,C_1,G,HB_1) >= 0
0.0833333 I01(B_0; HB_1 | C_1) >= 0
0.0833333 I01(B_0,inputs_and_H_4; HB_1 | A_0,A_1,B_1,H_4,G) >= 0
0.0555556 I01(B_0,inputs_and_H_4; HB_1 | A_0,A_1,B_1,H_4,C_0,G) >= 0
0.0277778 I01(B_0; HB_1 | A_0,B_1,H_4,C_0,G,HA_0) >= 0
0.0833333 I01(B_1; H_4 | A_0,B_0,C_1) >= 0
0.0833333 I01(B_1; C_1 | A_0) >= 0
0.0833333 I01(B_1; G | A_0,B_0,H_4,C_1) >= 0
0.0277778 I01(B_1,inputs_and_H_4; HA_0 | A_0,A_1,B_0,H_4,C_1,G) >= 0
0.305556 I01(A_1,B_0,inputs_and_H_4; C_0 | A_0,B_1,H_4,G,HA_0,HB_1) >= 0
0.0833333 I01(B_0,inputs_and_H_4; C_0 | A_0,A_1,B_1,H_4,G,HA_0,HB_1) >= 0
0.222222 I01(A_1,B_0,inputs_and_H_4; C_0 | A_0,B_1,H_4,C_1,G,HA_0,HB_1) >= 0
0.0833333 I01(B_0,inputs_and_H_4; G | A_0,A_1,B_1,H_4,C_1,HA_0,HB_1) >= 0
0.0833333 I01(A_1,B_0,inputs_and_H_4; HA_0 | A_0,B_1,H_4,C_0,G,HB_1) >= 0
0.0277778 I01(A_1,inputs_and_H_4; HB_1 | A_0,B_0,B_1,H_4,C_0,G,HA_0) >= 0
0.0833333 I01(C_0; C_1 | B_1) >= 0
0.0833334 I01(C_0; C_1 | A_0,B_1) >= 0
0.0277778 I01(C_0; C_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_0) >= 0
0.222222 I01(C_0; C_1 | A_0,B_1,H_4,G,HA_0,HB_1) >= 0
0.0833333 I01(C_1; HA_0 | A_0,A_1,B_1,H_4,G,HB_1) >= 0
0.0555555 I01(C_1; HA_0 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HB_1) >= 0
0.0555556 I01(C_1; HA_0 | A_0,A_1,B_1,H_4,C_0,G,HB_1) >= 0
0.0555555 I01(C_1; HB_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G) >= 0
0.0555556 I01(C_1; HB_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_0) >= 0
0.0833333 I10(A_0; A_1 | C_0) >= 0
0.0555556 I10(A_0; A_1 | B_0,C_0) >= 0
0.194444 I10(A_0; A_1 | B_0,B_1,C_1) >= 0
0.0833333 I10(A_0; A_1 | B_1,H_4,C_1,G,HA_1) >= 0
0.0833333 I10(A_0; B_0 | A_1,C_0) >= 0
0.166667 I10(A_0; B_0 | C_1) >= 0
0.0277778 I10(A_0; B_0 | A_1,C_1,G) >= 0
0.0277778 I10(A_0; B_0 | A_1,B_1,H_4,G,HA_1) >= 0
0.0277777 I10(A_0; B_0 | A_1,C_1,G,HA_1) >= 0
0.0833333 I10(A_0; B_1) >= 0
0.0277778 I10(A_0; B_1 | C_1,HA_1) >= 0
0.0555555 I10(A_0; B_1 | B_0,H_4,C_1,G,HB_0) >= 0
0.0833333 I10(A_0; H_4 | A_1,B_0,G) >= 0
0.0833333 I10(A_0; H_4 | B_1,C_1,HA_1) >= 0
0.0833333 I10(A_0; C_0) >= 0
0.0555556 I10(A_0; C_0 | B_0) >= 0
0.0277778 I10(A_0; C_0 | A_1,B_0,B_1,C_1) >= 0
0.0277778 I10(A_0; C_0 | A_1,B_0,H_4,G) >= 0
0.0277777 I10(A_0,inputs_and_H_4; C_0 | A_1,B_0,B_1,H_4,C_1,G,HA_1,HB_0) >= 0
0.0833334 I10(A_0; G | A_1) >= 0
0.0833333 I10(A_0; G | B_1,H_4,C_1,HA_1) >= 0
0.0277778 I10(A_0; HA_1 | C_1) >= 0
0.0555555 I10(A_0; HA_1 | B_1,C_1) >= 0
0.0277778 I10(A_0; HA_1 | A_1,H_4,C_0,G) >= 0
0.0277778 I10(A_0; HA_1 | A_1,B_0,H_4,C_0,G) >= 0
0.0277778 I10(A_0,inputs_and_H_4; HA_1 | A_1,B_0,B_1,H_4,C_0,G) >= 0
0.0833333 I10(A_0; HA_1 | A_1,B_0,H_4,C_0,C_1,G,HB_0) >= 0
0.0277778 I10(A_0; HB_0 | A_1,B_0,H_4,G,HA_1) >= 0
0.0277777 I10(A_0,inputs_and_H_4; HB_0 | A_1,B_0,B_1,H_4,C_1,G,HA_1) >= 0
0.0833333 I10(A_1; B_1 | A_0) >= 0
0.194444 I10(A_1; B_1 | B_0,C_1) >= 0
0.0555556 I10(A_1; B_1 | A_0,B_0,HB_0) >= 0
0.0555555 I10(A_1; H_4 | A_0,B_0,B_1,G,HB_0) >= 0
0.0277778 I10(A_1; H_4 | A_0,B_0,C_1,G,HB_0) >= 0
0.0277777 I10(A_1; C_1 | B_0) >= 0
0.0555555 I10(A_1,inputs_and_H_4; C_1 | A_0,B_0,B_1,H_4,C_0,G,HB_0) >= 0
0.0277778 I10(A_1; G | A_0,B_0,B_1,C_1) >= 0
0.0555555 I10(A_1; G | A_0,B_0,B_1,HB_0) >= 0
0.0555556 I10(A_1; HB_0 | A_0,B_0) >= 0
0.0277778 I10(A_1; HB_0 | A_0,B_0,C_1,G) >= 0
0.0833333 I10(B_0; B_1 | A_0,A_1) >= 0
0.194445 I10(B_0; B_1 | A_0,C_1) >= 0
```

```
0.0555555 I10(B_0; B_1 | A_0,A_1,H_4,C_1,G,HA_1) >= 0
0.0555555 I10(B_0; B_1 | H_4,C_1,G,HB_0) >= 0
0.0277777 I10(B_0; B_1 | A_1,H_4,C_0,G,HA_1,HB_0) >= 0
0.0277778 I10(B_0; H_4 | A_1,G) >= 0
0.0277778 I10(B_0; H_4 | A_0,A_1,B_1,C_1,G) >= 0
0.0277777 I10(B_0; H_4 | A_0,A_1,C_1,G,HA_1) >= 0
0.0555556 I10(B_0; C_1 | A_1,G) >= 0
0.0277778 I10(B_0,inputs_and_H_4; C_1 | A_0,A_1,B_1,H_4,G,HA_1,HB_0) >= 0
0.0833333 I10(B_0; G | A_0,A_1) >= 0
0.0277777 I10(B_0; HA_1 | A_1,C_1,G) >= 0
0.0555555 I10(B_1; H_4 | A_0,A_1,B_0) >= 0
0.0555555 I10(B_1; H_4 | C_1) >= 0
0.0277778 I10(B_1; H_4 | A_0,A_1,C_1,G) >= 0
0.0277778 I10(B_1; H_4 | A_0,A_1,B_0,G,HB_0) >= 0
0.0277778 I10(B_1; C_0 | A_1,B_0) >= 0
0.138889 I10(B_1; C_0 | A_0,A_1,B_0,C_1) >= 0
0.0277777 I10(B_1; C_0 | A_1,H_4,G,HA_1) >= 0
0.0277778 I10(B_1,inputs_and_H_4; C_0 | A_0,A_1,B_0,H_4,G,HA_1) >= 0
0.0277778 I10(B_1; G | A_0,A_1,B_0) >= 0
0.0555555 I10(B_1,inputs_and_H_4; G | A_0,A_1,B_0,H_4) >= 0
0.0277778 I10(B_1; G | A_0,B_0,C_1) >= 0
0.0555555 I10(B_1; G | H_4,C_1,HB_0) >= 0
0.0277778 I10(B_1; HA_1 | A_1,B_0,H_4,C_0,G) >= 0
0.0555555 I10(B_1; HB_0 | H_4,C_1) >= 0
0.0277778 I10(B_1; HB_0 | A_0,A_1,B_0,G) >= 0
0.0277777 I10(B_1; HB_0 | A_1,H_4,C_0,G,HA_1) >= 0
0.555556 I10(A_0,B_1,inputs_and_H_4; C_0 | A_1,B_0,H_4,G,HA_1,HB_0) >= 0
0.0555556 I10(B_0,B_1,inputs_and_H_4; HA_1 | A_0,A_1,H_4,C_0,C_1,G) >= 0
0.0833333 I10(B_1,inputs_and_H_4; HA_1 | A_0,A_1,B_0,H_4,C_0,C_1,G,HB_0) >= 0
0.111111 I10(A_0,B_1,inputs_and_H_4; HB_0 | A_1,B_0,H_4,C_0,G,HA_1) >= 0
0.0277778 I10(B_1,inputs_and_H_4; HB_0 | A_0,A_1,B_0,H_4,C_1,G,HA_1) >= 0
0.138889 I10(C_0; C_1 | A_0,A_1,B_0) >= 0
0.0277778 I10(C_0; C_1 | A_1,B_0,B_1) >= 0
0.0277778 I10(C_0; C_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G) >= 0
0.0277777 I10(C_0; C_1 | A_1,B_0,B_1,H_4,G,HA_1) >= 0
0.0555555 I10(C_0; C_1 | A_0,B_0,B_1,H_4,G,HB_0) >= 0
0.138889 I10(C_0; C_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_1,HB_0) >= 0
0.0277777 I10(C_0; HA_1 | A_1,H_4,G) >= 0
0.0277778 I10(C_0; HA_1 | A_1,B_0,H_4,G) >= 0
0.0277778 I10(C_0; HA_1 | A_0,A_1,H_4,C_1,G) >= 0
0.0277777 I10(C_1; HA_1 | A_0,A_1,H_4,C_0,G) >= 0
0.0277778 I10(C_1; HA_1 | A_0,A_1,B_0,H_4,G,HB_0) >= 0
0.0833333 I10(C_1; HA_1 | A_1,B_0,H_4,C_0,G,HB_0) >= 0
0.0277778 I10(C_1; HA_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HB_0) >= 0
0.0555556 I10(C_1; HB_0 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G) >= 0
0.0277778 I10(C_1; HB_0 | A_0,A_1,B_1,H_4,G,HA_1) >= 0
0.0277777 I10(C_1; HB_0 | A_1,B_0,B_1,H_4,C_0,G,HA_1) >= 0
0.0555556 I10(C_1; HB_0 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_1) >= 0
0.0416667 I11(A_0; A_1 | H_4) >= 0
0.0833333 I11(A_0; A_1 | B_1,C_0) >= 0
0.0416667 I11(A_0; A_1 | B_1,C_1) >= 0
0.0416667 I11(A_0; A_1 | B_0,C_0,C_1) >= 0
0.0416667 I11(A_0; A_1 | C_1,HA_1) >= 0
0.0416666 I11(A_0; A_1 | B_0,H_4,C_1,G,HA_1) >= 0
0.0416667 I11(A_0; A_1 | B_1,H_4,C_1,HB_1) >= 0
0.0416667 I11(A_0; A_1 | B_1,H_4,C_1,G,HB_1) >= 0
0.0416667 I11(A_0; A_1 | H_4,G,HA_1,HB_1) >= 0
0.0416667 I11(A_0; B_0 | A_1,B_1,C_0) >= 0
0.0416666 I11(A_0; B_0 | A_1,C_1) >= 0
0.0416667 I11(A_0; B_0 | A_1,B_1,H_4,C_1,G,HA_1) >= 0
0.0416667 I11(A_0; B_0 | A_1,B_1,H_4,HB_1) >= 0
0.0416667 I11(A_0; B_1 | A_1) >= 0
0.0416667 I11(A_0; B_1 | A_1,B_0,C_0) >= 0
0.0416666 I11(A_0; B_1 | C_1) >= 0
0.0416667 I11(A_0; B_1 | A_1,B_0,C_1) >= 0
0.0833334 I11(A_0; B_1 | B_0,C_0,C_1) >= 0
0.0416667 I11(A_0; B_1 | A_1,B_0,H_4,C_1,HA_1) >= 0
0.0416667 I11(A_0; B_1 | A_1,H_4,G,HA_1,HB_1) >= 0
```

```
0.0833334 I11(A_0; H_4) >= 0
0.0416668 I11(A_0; H_4 | A_1,C_1,HA_1) >= 0
0.0416666 I11(A_0; H_4 | B_0,C_1,G,HA_1) >= 0
0.0416667 I11(A_0; C_0 | A_1,B_1) >= 0
0.125 I11(A_0; C_0 | B_0,C_1) >= 0
0.611111 I11(A_0; C_0 | A_1,B_1,H_4,G,HA_1,HB_1) >= 0
0.0416667 I11(A_0; G | H_4) >= 0
0.0416666 I11(A_0; G | B_0,C_1,HA_1) >= 0
0.0416667 I11(A_0; HA_1 | C_1) >= 0
0.0416666 I11(A_0; HA_1 | B_0,C_1) >= 0
0.0416667 I11(A_0; HA_1 | H_4,G) >= 0
0.0416668 I11(A_0,inputs_and_H_4; HB_1 | A_1,B_0,B_1,H_4,C_1,HA_1) >= 0
0.0416667 I11(A_0; HB_1 | H_4,G,HA_1) >= 0
0.0833333 I11(A_0; HB_1 | A_1,B_1,H_4,C_0,G,HA_1) >= 0
0.0833334 I11(A_1; B_0) >= 0
0.0833333 I11(A_1; B_0 | A_0,B_1,C_0) >= 0
0.0833333 I11(A_1; B_0 | C_1) >= 0
0.0416667 I11(A_1; B_0 | A_0,B_1,C_1) >= 0
0.0416667 I11(A_1; B_0 | A_0,C_1,HB_1) >= 0
0.0833334 I11(A_1; H_4 | B_1,C_1,HB_1) >= 0
0.0833333 I11(A_1; C_1 | A_0,B_0,B_1,C_0) >= 0
0.0416667 I11(A_1; G | B_1,H_4,C_1,HB_1) >= 0
0.0833333 I11(A_1; HB_1 | B_1,C_1) >= 0
0.0833333 I11(B_0; B_1 | A_1,C_0) >= 0
0.0416667 I11(B_0; B_1 | A_0,C_1) >= 0
0.0416667 I11(B_0; B_1 | A_1,C_1) >= 0
0.0833334 I11(B_0; B_1 | C_0,C_1) >= 0
0.0416667 I11(B_0; B_1 | A_1,H_4,HA_1) >= 0
0.0416667 I11(B_0; B_1 | A_1,H_4,C_1,G,HA_1) >= 0
0.0416667 I11(B_0; B_1 | A_0,A_1,H_4,C_1,HB_1) >= 0
0.0416667 I11(B_0; B_1 | A_1,H_4,C_1,HA_1,HB_1) >= 0
0.0416667 I11(B_0; H_4 | A_0,A_1,B_1) >= 0
0.0416667 I11(B_0; H_4 | A_0,A_1,B_1,C_1) >= 0
0.0416667 I11(B_0; H_4 | A_1,HA_1) >= 0
0.0416667 I11(B_0; H_4 | A_0,A_1,C_1,HB_1) >= 0
0.0833333 I11(B_0; C_0 | A_1) >= 0
0.0833333 I11(B_0; C_0 | C_1) >= 0
0.611111 I11(B_0,inputs_and_H_4; C_0 | A_0,A_1,B_1,H_4,G,HA_1,HB_1) >= 0
0.0416667 I11(B_0; HA_1 | A_1) >= 0
0.0416667 I11(B_0; HB_1 | A_1,B_1,H_4) >= 0
0.0416667 I11(B_0; HB_1 | A_0,C_1) >= 0
0.0416667 I11(B_0; HB_1 | A_1,H_4,C_1,HA_1) >= 0
0.0416667 I11(B_1; H_4 | A_0,A_1) >= 0
0.0416667 I11(B_1; H_4 | A_0,A_1,C_1,HA_1) >= 0
0.0416667 I11(B_1; C_1 | A_0,A_1,B_0,C_0) >= 0
0.0416667 I11(B_1; C_1 | A_1,H_4,G,HA_1,HB_1) >= 0
0.0416667 I11(B_1,inputs_and_H_4; G | A_0,A_1,B_0,H_4,C_1,HA_1) >= 0
0.0416667 I11(B_1; HA_1 | A_1,H_4) >= 0
0.0416667 I11(B_1; HA_1 | A_0,A_1,C_1) >= 0
0.0416667 I11(A_0,B_0,B_1,inputs_and_H_4; G | A_1,H_4,HA_1) >= 0
0.0416667 I11(B_0,inputs_and_H_4; G | A_0,A_1,B_1,H_4,HB_1) >= 0
0.0416667 I11(A_1,B_0,inputs_and_H_4; G | A_0,B_1,H_4,C_1,HB_1) >= 0
0.0416667 I11(A_0,B_0,inputs_and_H_4; G | A_1,B_1,H_4,C_1,HA_1,HB_1) >= 0
0.0416667 I11(A_0,inputs_and_H_4; HA_1 | A_1,B_0,B_1,H_4) >= 0
0.0416667 I11(B_0,inputs_and_H_4; HA_1 | A_0,A_1,B_1,H_4,C_1) >= 0
0.0833333 I11(A_0,B_0,inputs_and_H_4; HA_1 | A_1,B_1,H_4,C_0,G,HB_1) >= 0
0.0833333 I11(B_0,inputs_and_H_4; HB_1 | A_0,A_1,B_1,H_4,C_0,G,HA_1) >= 0
0.0416667 I11(B_0,inputs_and_H_4; HB_1 | A_0,A_1,B_1,H_4,C_1,G,HA_1) >= 0
0.0416667 I11(C_0; C_1 | A_1,B_0) >= 0
0.0833334 I11(C_0; C_1 | B_1) >= 0
0.0416667 I11(C_0; C_1 | A_0,A_1,B_0,B_1) >= 0
0.0416666 I11(C_0; C_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HB_1) >= 0
0.208333 I11(C_0; C_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_1,HB_1) >= 0
0.0416667 I11(C_1; HA_1 | A_0,A_1,B_1,H_4,G,HB_1) >= 0
0.0416666 I11(C_1; HA_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HB_1) >= 0
0.0416667 I11(C_1; HB_1 | A_1,H_4,G,HA_1) >= 0
0.0416667 I11(C_1; HB_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_1) >= 0
0.0833333 I11(C_1; HB_1 | A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_1) >= 0
```

```
-HR(inputs_and_H_4) - HR(HA_0) - HR(HA_1) - HR(HB_0) - HR(HB_1) + HR(inputs_and_H_4,HA_0,HA_1,HB_0,HB_1) >= 0
0.0833333 H00(A_0) == 0.0833333
0.0833333 H01(A_0) == 0.0833333
-0.0833333 H10(A_0) == -0.0833333
-0.0833333 H11(A_0) == -0.0833333
-0.0833333 H00(A_1) == -0.0833333
-0.0833333 H01(A_1) == -0.0833333
0.0833333 H10(A_1) == 0.0833333
0.0833333 H11(A_1) == 0.0833333
0.0833333 H00(B_0) == 0.0833333
-0.0833333 H01(B_0) == -0.0833333
0.0833333 H10(B_0) == 0.0833333
-0.0833333 H11(B_0) == -0.0833333
-0.0833333 H00(B_1) == -0.0833333
0.0833333 H01(B_1) == 0.0833333
-0.0833333 H10(B_1) == -0.0833333
0.0833333 H11(B_1) == 0.0833333
0.25 H00(C_1) == 0.25
0.25 H01(C_1) == 0.25
0.25 H10(C_1) == 0.25
0.25 H11(C_1) == 0.25
-0.416667 H00(A_0,B_0,C_0) - 0.416667 H00(A_1,B_1,C_1) + 0.416667 H00(A_0,A_1,B_0,B_1,C_0,C_1) == 0
0.527778 H00(A_0,B_0,H_4,G,HA_0,HB_0) - 0.527778 H00(A_0,B_0,H_4,C_0,G,HA_0,HB_0) >= 0
0.25 H00(A_0,B_0,C_0) - 0.25 H00(A_0,A_1,B_0,B_1,C_0,C_1) - 0.25 H00(A_0,B_0,H_4,C_0,G,HA_0,HB_0) + 0.25 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_0,HB_0) >= 0
0.0555556 eps_W + 0.111111 eps - 0.0555556 HR(G,inputs_and_H_4) + 0.0555556 HR(G,inputs_and_H_4,HA_0,HB_0) + 0.0555556 HR(G,inputs_and_H_4,C_0) - 0.0555556 HR(G,inputs_and_H_4,HA_0,HB_0,C_0) + 0.0555556 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G) - 0.0555556 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G) - 0.0555556 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_0,HB_0) + 0.0555556 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_0,HB_0) >= 0
0.0277778 eps_W + 0.0555556 eps - 0.0277778 HR(G,inputs_and_H_4,HA_0) + 0.0277778 HR(G,inputs_and_H_4,HA_0,HB_0,C_1) + 0.0277778 HR(G,inputs_and_H_4,HA_0,C_0,C_1) - 0.0277778 HR(G,inputs_and_H_4,HA_0,HB_0,C_0,C_1) + 0.0277778 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_0) - 0.0277778 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_0) - 0.0277778 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_1,G,HA_0,HB_0) + 0.0277778 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_0,HB_0) >= 0
0.0277778 eps_W + 0.0555555 eps - 0.0277778 HR(G,inputs_and_H_4,HB_0) + 0.0277778 HR(G,inputs_and_H_4,HA_0,HB_0) + 0.0277778 HR(G,inputs_and_H_4,HB_0,C_0) - 0.0277778 HR(G,inputs_and_H_4,HA_0,HB_0,C_0) + 0.0277778 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HB_0) - 0.0277778 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HB_0) - 0.0277778 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_0,HB_0) + 0.0277778 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_0,HB_0) >= 0
0.0555556 eps_W + 0.111111 eps - 0.0555556 HR(G,inputs_and_H_4,HB_0) + 0.0555556 HR(G,inputs_and_H_4,HA_0,HB_0,C_1) + 0.0555556 HR(G,inputs_and_H_4,HB_0,C_0,C_1) - 0.0555556 HR(G,inputs_and_H_4,HA_0,HB_0,C_0,C_1) + 0.0555556 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HB_0) - 0.0555556 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HB_0) - 0.0555556 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_1,G,HA_0,HB_0) + 0.0555556 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_0,HB_0) >= 0
0.361111 eps_W + 0.361111 eps + 0.361111 HR(G,inputs_and_H_4,HA_0,HB_0) - 0.361111 HR(G,inputs_and_H_4,HA_0,HB_0,C_0) - 0.361111 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_0,HB_0) + 0.361111 H00(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_0,HB_0) >= 0
-0.416667 H01(A_0,B_1,C_0) - 0.416667 H01(A_1,B_0,C_1) + 0.416667 H01(A_0,A_1,B_0,B_1,C_0,C_1) == 0
0.527778 H01(A_0,B_1,H_4,G,HA_0,HB_1) - 0.527778 H01(A_0,B_1,H_4,C_0,G,HA_0,HB_1) >= 0
0.25 H01(A_0,B_1,C_0) - 0.25 H01(A_0,A_1,B_0,B_1,C_0,C_1) - 0.25 H01(A_0,B_1,H_4,C_0,G,HA_0,HB_1) + 0.25 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_0,HB_1) >= 0
0.0555556 eps_W + 0.111111 eps - 0.0555556 HR(G,inputs_and_H_4) + 0.0555556 HR(G,inputs_and_H_4,HA_0,HB_1) + 0.0555556 HR(G,inputs_and_H_4,C_1) - 0.0555556 HR(G,inputs_and_H_4,HA_0,HB_1,C_1) + 0.0555556 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G) - 0.0555556 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G) - 0.0555556 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_0,HB_1) + 0.0555556 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_0,HB_1) >= 0
0.0277778 eps_W + 0.0555555 eps - 0.0277778 HR(G,inputs_and_H_4,HA_0) + 0.0277778 HR(G,inputs_and_H_4,HA_0,HB_1,C_0) + 0.0277778 HR(G,inputs_and_H_4,HA_0,C_0,C_1) - 0.0277778 HR(G,inputs_and_H_4,HA_0,HB_1,C_0,C_1) + 0.0277778 H01(A_0,A_1,B_0,B_1,H_4,
```

```
    inputs_and_H_4,G,HA_0) - 0.0277778 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_0
    ) - 0.0277778 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_1,G,HA_0,HB_1) + 0.0277778 H01(
    A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_0,HB_1) >= 0
458 0.0277778 eps_W + 0.0555556 eps - 0.0277778 HR(G,inputs_and_H_4,HB_1) + 0.0277778 HR(G,
    inputs_and_H_4,HA_0,HB_1) + 0.0277778 HR(G,inputs_and_H_4,HB_1,C_1) - 0.0277778 HR(G,
    inputs_and_H_4,HA_0,HB_1,C_1) + 0.0277778 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HB_1)
     - 0.0277778 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HB_1) - 0.0277778 H01(A_0,A_1,
    B_0,B_1,H_4,inputs_and_H_4,G,HA_0,HB_1) + 0.0277778 H01(A_0,A_1,B_0,B_1,H_4,
    inputs_and_H_4,C_0,G,HA_0,HB_1) >= 0
459 0.0555556 eps_W + 0.111111 eps - 0.0555556 HR(G,inputs_and_H_4,HB_1) + 0.0555556 HR(G,
    inputs_and_H_4,HA_0,HB_1,C_0) + 0.0555556 HR(G,inputs_and_H_4,HB_1,C_0,C_1) - 0.0555556
    HR(G,inputs_and_H_4,HA_0,HB_1,C_0,C_1) + 0.0555556 H01(A_0,A_1,B_0,B_1,H_4,
    inputs_and_H_4,G,HB_1) - 0.0555556 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HB_1
    ) - 0.0555556 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_1,G,HA_0,HB_1) + 0.0555556 H01(
    A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_0,HB_1) >= 0
460 0.361111 eps_W + 0.361111 eps + 0.361111 HR(G,inputs_and_H_4,HA_0,HB_1) - 0.361111 HR(G,
    inputs_and_H_4,HA_0,HB_1,C_1) - 0.361111 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_0,
    HB_1) + 0.361111 H01(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_0,HB_1) >= 0
461 -0.416667 H10(A_1,B_0,C_0) - 0.416667 H10(A_0,B_1,C_1) + 0.416667 H10(A_0,A_1,B_0,B_1,C_0,
    C_1) == 0
462 0.527778 H10(A_1,B_0,H_4,G,HA_1,HB_0) - 0.527778 H10(A_1,B_0,H_4,C_0,G,HA_1,HB_0) >= 0
463 0.25 H10(A_1,B_0,C_0) - 0.25 H10(A_0,A_1,B_0,B_1,C_0,C_1) - 0.25 H10(A_1,B_0,H_4,C_0,G,HA_1,
    HB_0) + 0.25 H10(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_1,HB_0) >= 0
464 0.0833333 eps_W + 0.166667 eps - 0.0833333 HR(G,inputs_and_H_4) + 0.0833333 HR(G,
    inputs_and_H_4,HA_1,HB_0,C_0) + 0.0833333 HR(G,inputs_and_H_4,C_0,C_1) - 0.0833333 HR(G,
    inputs_and_H_4,HA_1,HB_0,C_0,C_1) + 0.0833333 H10(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G)
    - 0.0833333 H10(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G) - 0.0833333 H10(A_0,A_1,
    B_0,B_1,H_4,inputs_and_H_4,C_1,G,HA_1,HB_0) + 0.0833333 H10(A_0,A_1,B_0,B_1,H_4,
    inputs_and_H_4,C_0,C_1,G,HA_1,HB_0) >= 0
465 0.0833333 eps_W + 0.166667 eps - 0.0833333 HR(G,inputs_and_H_4,HB_0) + 0.0833333 HR(G,
    inputs_and_H_4,HA_1,HB_0) + 0.0833333 HR(G,inputs_and_H_4,HB_0,C_1) - 0.0833333 HR(G,
    inputs_and_H_4,HA_1,HB_0,C_1) + 0.0833333 H10(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HB_0)
     - 0.0833333 H10(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HB_0) - 0.0833333 H10(A_0,A_1,
    B_0,B_1,H_4,inputs_and_H_4,G,HA_1,HB_0) + 0.0833333 H10(A_0,A_1,B_0,B_1,H_4,
    inputs_and_H_4,C_0,G,HA_1,HB_0) >= 0
466 0.361111 eps_W + 0.361111 eps + 0.361111 HR(G,inputs_and_H_4,HA_1,HB_0) - 0.361111 HR(G,
    inputs_and_H_4,HA_1,HB_0,C_1) - 0.361111 H10(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_1,
    HB_0) + 0.361111 H10(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_1,HB_0) >= 0
467 -0.416667 H11(A_1,B_1,C_0) - 0.416667 H11(A_0,B_0,C_1) + 0.416667 H11(A_0,A_1,B_0,B_1,C_0,
    C_1) == 0
468 0.527778 H11(A_1,B_1,H_4,G,HA_1,HB_1) - 0.527778 H11(A_1,B_1,H_4,C_0,G,HA_1,HB_1) >= 0
469 0.25 H11(A_1,B_1,C_0) - 0.25 H11(A_0,A_1,B_0,B_1,C_0,C_1) - 0.25 H11(A_1,B_1,H_4,C_0,G,HA_1,
    HB_1) + 0.25 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_1,HB_1) >= 0
470 0.0833333 eps_W + 0.166667 eps - 0.0833333 HR(G,inputs_and_H_4,HA_1) + 0.0833333 HR(G,
    inputs_and_H_4,HA_1,HB_1,C_1) + 0.0833333 HR(G,inputs_and_H_4,HA_1,C_0,C_1) - 0.0833333
    HR(G,inputs_and_H_4,HA_1,HB_1,C_0,C_1) + 0.0833333 H11(A_0,A_1,B_0,B_1,H_4,
    inputs_and_H_4,G,HA_1) - 0.0833333 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_1
    ) - 0.0833333 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_1,G,HA_1,HB_1) + 0.0833333 H11(
    A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,C_1,G,HA_1,HB_1) >= 0
471 0.0833333 eps_W + 0.166667 eps - 0.0833333 HR(G,inputs_and_H_4,HB_1) + 0.0833333 HR(G,
    inputs_and_H_4,HA_1,HB_1) + 0.0833333 HR(G,inputs_and_H_4,HB_1,C_0) - 0.0833333 HR(G,
    inputs_and_H_4,HA_1,HB_1,C_0) + 0.0833333 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HB_1)
     - 0.0833333 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HB_1) - 0.0833333 H11(A_0,A_1,
    B_0,B_1,H_4,inputs_and_H_4,G,HA_1,HB_1) + 0.0833333 H11(A_0,A_1,B_0,B_1,H_4,
    inputs_and_H_4,C_0,G,HA_1,HB_1) >= 0
472 0.361111 eps_W + 0.361111 eps + 0.361111 HR(G,inputs_and_H_4,HA_1,HB_1) - 0.361111 HR(G,
    inputs_and_H_4,HA_1,HB_1,C_0) - 0.361111 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,G,HA_1,
    HB_1) + 0.361111 H11(A_0,A_1,B_0,B_1,H_4,inputs_and_H_4,C_0,G,HA_1,HB_1) >= 0
473 0.111111 eps_W + 0.111111 eps - 0.0555556 H00(H_4,G) + 0.0555556 H00(A_0,B_0,H_4,C_0,G) +
    0.0555556 H01(H_4,G) - 0.0555556 H01(A_0,B_0,H_4,C_1,G) >= 0
474 0.111111 eps_W + 0.111111 eps + 0.0555556 H00(H_4,G) - 0.0555556 H00(A_0,B_1,H_4,C_1,G) -
    0.0555556 H01(H_4,G) + 0.0555556 H01(A_0,B_1,H_4,C_0,G) >= 0
475 0.0555556 eps_W + 0.0555556 eps - 0.0277778 H00(H_4,G,HA_0) + 0.0277778 H00(A_0,B_0,H_4,C_0,
    G,HA_0) + 0.0277778 H01(H_4,G,HA_0) - 0.0277778 H01(A_0,B_0,H_4,C_1,G,HA_0) >= 0
476 0.0555556 eps_W + 0.0555556 eps + 0.0277778 H00(H_4,G,HA_0) - 0.0277778 H00(A_0,B_1,H_4,C_1,
    G,HA_0) - 0.0277778 H01(H_4,G,HA_0) + 0.0277778 H01(A_0,B_1,H_4,C_0,G,HA_0) >= 0
477 0.166667 eps_W + 0.166667 eps - 0.0833333 H00(H_4,G,HB_0) + 0.0833333 H00(A_0,B_0,H_4,C_0,G,
    HB_0) + 0.0833333 H10(H_4,G,HB_0) - 0.0833333 H10(A_0,B_0,H_4,C_1,G,HB_0) >= 0
```

```
0.166667 eps_W + 0.166667 eps + 0.0833333 H00(H_4,G,HB_0) - 0.0833333 H00(A_1,B_0,H_4,C_1,G,
    HB_0) - 0.0833333 H10(H_4,G,HB_0) + 0.0833333 H10(A_1,B_0,H_4,C_0,G,HB_0) >= 0
0.333333 eps_W + 0.333333 eps - 0.166667 H00(A_0,B_0,C_1) + 0.166667 H11(A_0,B_0,C_1) >= 0
0.333333 eps_W + 0.333333 eps + 0.166667 H00(A_1,B_1,C_1) - 0.166667 H11(A_1,B_1,C_1) >= 0
0.333333 eps_W + 0.333333 eps + 0.166667 H01(A_1,B_0,C_1) - 0.166667 H10(A_1,B_0,C_1) >= 0
0.333333 eps_W + 0.333333 eps - 0.166667 H01(A_0,B_1,C_1) + 0.166667 H10(A_0,B_1,C_1) >= 0
0.166667 eps_W + 0.166667 eps - 0.0833333 H01(H_4,G,HB_1) + 0.0833333 H01(A_0,B_1,H_4,C_0,G,
    HB_1) + 0.0833333 H11(H_4,G,HB_1) - 0.0833333 H11(A_0,B_1,H_4,C_1,G,HB_1) >= 0
0.166667 eps_W + 0.166667 eps + 0.0833333 H01(H_4,G,HB_1) - 0.0833333 H01(A_1,B_1,H_4,C_1,G,
    HB_1) - 0.0833333 H11(H_4,G,HB_1) + 0.0833333 H11(A_1,B_1,H_4,C_0,G,HB_1) >= 0
0.166667 eps_W + 0.166667 eps - 0.0833333 H10(H_4,G,HA_1) + 0.0833333 H10(A_1,B_0,H_4,C_0,G,
    HA_1) + 0.0833333 H11(H_4,G,HA_1) - 0.0833333 H11(A_1,B_0,H_4,C_1,G,HA_1) >= 0
0.166667 eps_W + 0.166667 eps + 0.0833333 H10(H_4,G,HA_1) - 0.0833333 H10(A_1,B_1,H_4,C_1,G,
    HA_1) - 0.0833333 H11(H_4,G,HA_1) + 0.0833333 H11(A_1,B_1,H_4,C_0,G,HA_1) >= 0

 => 4.27351e-18 eps_H + 4.77778 eps_W + 5.44444 eps - HR(inputs_and_H_4) + HR(G,
     inputs_and_H_4) >= 1
```